10 Modellazione

OBIETTIVI FORMATIVI

- Conoscere le varie tecniche di modellazione classica
- Conoscere il linguaggio di modellazione unificato (UML) e i suoi principali tipi di diagramma
- Conoscere la terminologia dell'orientamento agli oggetti

Nel corso di un progetto di sviluppo software, e in particolare durante l'ingegneria dei requisiti e la progettazione, vengono utilizzate molte notazioni di modellazione. Questi vanno da schizzi molto informali di funzioni di sistema o layout dello schermo, a descrizioni altamente formali del comportamento del sistema. Le notazioni più comuni utilizzate sono i diagrammi box-and-line con semantica semiformale. In questo capitolo, discutiamo alcune di queste notazioni diagrammatiche.

Durante lo sviluppo del software, avviene molta comunicazione. Questa comunicazione è supportata da tutti i tipi di notazioni per trasmettere il suo messaggio. Uno schizzo di un layout dello schermo può supportare la comunicazione tra un utente e un tecnico dei requisiti. Una descrizione molto più formale delle interfacce di classe può supportare la comunicazione tra un progettista e uno sviluppatore.

Le notazioni più comuni utilizzate per supportare la comunicazione tra i vari stakeholder coinvolti nello sviluppo del software coinvolgono una sorta di diagrammi box-and-line. A volte, questi diagrammi hanno una semantica molto informale. Ad esempio, le caselle possono indicare parti del sistema, in cui non è chiaro cosa sia esattamente una parte. Una casella può indicare un sottosistema principale, un'altra casella può indicare l'insieme delle misure di sicurezza adottate. Allo stesso modo, le linee possono denotare una relazione parti di una relazione, una relazione chiamante, una relazione utilizza e così via. Mentre vengono disegnate, queste semantiche libere potrebbero non essere un problema. Ma il giorno dopo, sorgerà la confusione.

Un'alternativa è usare diagrammi che hanno una semantica più precisa. Se i lettori conoscono questa semantica, non c'è bisogno di confusione su cosa si intenda esattamente. Se la semantica è sufficientemente precisa, i diagrammi possono essere sottoposti a tutti i tipi di controlli di coerenza. Gli strumenti possono generare i diagrammi, leggerli ed eventualmente anche generare codice eseguibile dai diagrammi. Infine, possiamo collegare i diagrammi ai metodi che li generano, dando così una certa operazionalizzazione al loro processo di costruzione. Ad esempio, i passaggi in un metodo di progettazione possono essere collegati ai passaggi per generare il diagramma corrispondente. Questi ultimi tipi di diagrammi sono discussi nel Capitolo 12, insieme a una discussione sui metodi di progettazione corrispondenti.

Capitolo **Errore. Per applicare Heading 1 al testo da visualizzare in questo punto, utilizzare la scheda Home. Errore. Per applicare Heading 1 al testo da visualizzare in questo punto, utilizzare la scheda Home.**

In questo capitolo, discutiamo una serie di notazioni di modellazione semiformali. La maggior parte di loro utilizza una sorta di diagramma a scatole e linee. Alcuni usano un layout più testuale. I diagrammi e lo schema vengono generalmente disegnati durante l'ingegneria dei requisiti e la progettazione. Alcuni servono principalmente l'ingegneria dei requisiti. Ad esempio, i diagrammi dei casi d'uso vengono generalmente disegnati durante l'ingegneria dei requisiti. I diagrammi della struttura Jackson, d'altra parte, vengono utilizzati principalmente durante la progettazione. Molte notazioni di modellazione servono entrambi.

Attualmente, le notazioni di modellazione tradizionali derivano dall'Unified Modeling Language (UML). Molti diagrammi di UML, tuttavia, sono basati o derivati da quelli precedenti. E, certamente nelle applicazioni legacy, potresti imbatterti in molte di queste vecchie notazioni. Pertanto, in questo capitolo viene fornito un campione di entrambi.

UML si è evoluto da alcuni dei principali metodi di analisi e progettazione orientati agli oggetti. Il concetto di orientamento agli oggetti, a sua volta, ha le sue radici nello sviluppo dei linguaggi di programmazione, in particolare SIMULA-67 e Smalltalk. Per quanto riguarda l'analisi e la progettazione (e l'analisi dei requisiti), l'orientamento agli oggetti viene visualizzato al meglio evidenziando le differenze con i metodi di progettazione più tradizionali come la scomposizione funzionale e la progettazione del flusso di dati. Mentre le tecniche tradizionali si concentrano sull'identificazione delle *funzioni* che il sistema deve svolgere, i metodi orientati agli oggetti si concentrano sull'identificazione e sull'interrelazione degli *oggetti* che svolgono un ruolo nel sistema. La sezione 10.2 introduce una serie di concetti rilevanti, come oggetto, attributo, classe, relazione. In un modo o nell'altro, questi concetti vengono visualizzati in molti tipi di diagramma UML.

10.1 TECNICHE DI MODELLAZIONE CLASSICA

Discutiamo di quattro tecniche di modellazione classiche che esistono da un po' di tempo:

- La modellazione entità-relazione (ERM) è una tecnica di modellazione dei dati, sperimentata da Chen (**1976**). I diagrammi di classe UML sono basati su ERM.
- Le macchine a stati finiti (FSM) vengono utilizzate per modellare stati e transizioni di stato. All'inizio, alcuni tipi di linguaggio formale, come quelli usati nei compilatori, erano modellati come macchine a stati finiti. I diagrammi delle macchine a stati UML si basano su macchine a stati finiti.
- I diagrammi di flusso di dati (DFD) modellano un sistema come un insieme di processi e flussi di dati che collegano questi processi. È la notazione utilizzata nella progettazione del flusso di dati. I diagrammi di sequenza UML assomigliano ai DFD.
- Le carte CRC sono un semplice strumento di elicitazione dei requisiti. Gran parte delle informazioni raccolte sulle schede CRC possono anche essere rappresentate come diagrammi di comunicazione UML.

Molte altre notazioni di modellazione classiche sono legate a un determinato metodo di analisi o progettazione. Ne discuteremo alcuni nel **Capitolo 12** nel contesto di una

discussione sui metodi di progettazione. Quelli discussi qui sono relativamente indipendenti dal metodo.

10.1.1 Modellazione Entità-Relazione

Nei sistemi ad alta intensità di dati, la modellazione della (struttura dei) dati è una preoccupazione importante. Fino agli anni '70, le tecniche di modellazione dei dati confondevano molto le preoccupazioni di implementazione con le preoccupazioni derivanti dalla struttura logica dell'universo del discorso (UoD). Ad esempio, il catalogo del libro verrebbe modellato come una "tabella" contenente "tuple" ("record") con campi alfanumerici contenenti il titolo e l'autore e campi numerici contenenti l'anno di pubblicazione e il numero di pagine.

La modellazione entità-relazione (ERM), come sperimentata da Chen, è diretta a modellare la struttura logica e semantica dell'UoD, piuttosto che la sua realizzazione in un sistema di database. I modelli entità-relazione sono rappresentati nei **diagrammi entità-relazione** (ERD). Esistono molte varianti di ERM, che differiscono nelle loro notazioni grafiche ed estensioni all'approccio originale di Chen. Gli ingredienti di base dell'ERM sono riportati nella **Tabella 10.1** .

Table 10-1 Concetti ERM e loro significato

Entità	oggetto distinguibile di qualche tipo
Tipo di entità	tipo di un insieme di entità
Valore attributo	informazione (parzialmente) che descrive un'entità
Attributo	tipo di un insieme di valori di attributo
Relazione	associazione tra due o più entità

Un'entità è una "cosa" che può essere identificata in modo univoco. Le entità sono solitamente rappresentate in un ERD come rettangoli. Le entità di esempio sono:

- oggetti tangibili, come copie di un libro, identificati da un numero;
- oggetti immateriali, come libri identificati dal loro ISBN, o membri di qualche struttura organizzativa, come i dipendenti della biblioteca identificati dal loro numero di dipendente.

Le entità hanno proprietà note come attributi. Ad esempio, alcuni impiegati della biblioteca potrebbero avere il nome "Jones". Qui, 'Jones' è il valore dell'attributo chiamato 'nome'. Gli attributi sono solitamente rappresentati come cerchi o ellissi.

Sia le entità che i valori degli attributi hanno un **tipo.** Come modellatori, tendiamo a vedere un tipo come un insieme di proprietà condivise dalle sue istanze. Come implementatori, tendiamo a vedere un tipo come un insieme di valori con un numero di operazioni associate. Per l'attributo 'numero di libri in prestito', l'insieme di valori potrebbe essere 0..10 con operazioni come incremento e decremento. Per il tipo di entità "copia libro", le operazioni candidate sarebbero "prestito", "ritorno" e così via.

Le entità sono collegate attraverso le relazioni. Ad esempio, la relazione "prestito" coinvolge le entità "copia libro" e "membro biblioteca". Molto spesso, una relazione è binaria, cioè collega due entità. Una relazione è denotata da un diamante legato alle entità coinvolte.

I modelli entità-relazione impongono restrizioni alla cardinalità delle relazioni. Nella sua forma più semplice, le relazioni sono 1–1, 1–N o N–M. La relazione "prestito" è 1–N: una copia di un libro può essere presa in prestito da un solo membro, mentre un membro può aver preso in prestito più di una copia del libro. In un ERD, questi vincoli di cardinalità sono spesso indicati da piccoli ornamenti delle frecce che collegano le entità.

Un esempio di diagramma entità-relazione è riportato nella **Figura 10.1** . I vincoli di cardinalità sono stati indicati indicando esplicitamente l'insieme delle possibilità. Pertanto, questo ERD afferma che una copia del libro può essere presa in prestito al massimo da un membro e un membro può prendere in prestito fino a dieci copie del libro.

Un modello entità-relazione può essere ottenuto utilizzando una qualsiasi delle tecniche di elicitazione discusse nel **Capitolo 9** . In particolare, vengono spesso utilizzate l'analisi dei moduli e l'analisi delle descrizioni del linguaggio naturale. Poiché gli ERM raccontano solo una parte della storia, è necessario utilizzare tecniche aggiuntive per modellare altri aspetti. Molte tecniche di analisi strutturata, ad esempio, incorporano l'ERM per modellare l'aspetto dei dati.

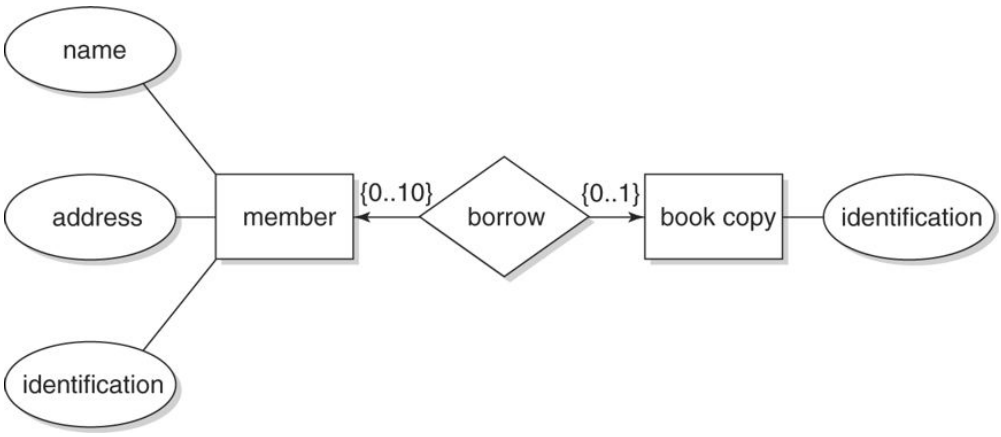


Figura 10.1 Un diagramma entità-relazione

La modellazione entità-relazione è una naturale conseguenza della modellazione di database. In origine, ERM aveva lo scopo di modellare la struttura logica dei dati, piuttosto che la struttura logica dell'UoD. Nell'euristica su come ottenere un modello entità-relazione "buono", queste radici sono ancora visibili. Ad esempio, alcune di queste euristiche assomigliano ai vincoli di normalizzazione della teoria dei database. Questo potrebbe spiegare perché alcuni non elogiano la modellazione entità-relazione come tecnica di specifica dei requisiti (si veda, ad esempio, (Davis, **1993**)).

L'attuale ERM ha molto in comune con le tecniche di analisi orientata agli oggetti. Ad esempio, le relazioni sottotipo-supertipo tra tipi di entità sono incluse in molte tecniche ERM. Al contrario, il diagramma delle classi di UML (vedi **Sezione 10.3.1**) include molti elementi di ERM.

10.1.2 Macchine a stati finiti

In qualsiasi momento, il nostro sistema bibliotecario si trova in uno di un (vasto) numero di possibili stati. Lo stato della biblioteca può essere espresso in termini di:

- la raccolta di titoli disponibile,
- la raccolta dei titoli ordinati ma non ancora ricevuti,
- la raccolta dei membri della biblioteca,
- il saldo del conto da cui vengono pagate le acquisizioni.

Qualsiasi azione intervenuta in biblioteca, sia essa la restituzione di un libro o la nomina di un nuovo dipendente, trasforma lo stato attuale in *un* nuovo stato.

Le tecniche di specifica dei requisiti che modellano un sistema in termini di stati e transizioni tra stati sono chiamate tecniche di modellazione **basate sullo stato.** Un formalismo semplice ma potente per specificare stati e transizioni di stato è la **macchina a stati finiti** (FSM). Un FSM consiste in un numero finito di stati e un insieme di transizioni da uno stato all'altro che si verificano sui segnali di ingresso da un insieme finito di possibili stimoli. Lo stato iniziale è uno stato appositamente designato da cui si avvia la macchina. Di solito, uno o più stati sono designati come stati finali. Pittoricamente, gli FSM sono rappresentati come **diagrammi di transizione di stato** (STD). In un diagramma di transizione di stato, gli stati sono rappresentati come bolle, con un'etichetta che identifica lo stato, e le transizioni sono indicate come archi etichettati da uno stato all'altro, dove l'etichetta indica lo stimolo che attiva la transizione. La Figura 10.2 fornisce un FSM che descrive i possibili stati di una copia di un libro e le transizioni tra questi stati. Lo stato finale è quello etichettato come "cancellato". Uno qualsiasi degli altri potrebbe essere designato come stato iniziale.

Figura 10.2 modella solo una piccola parte del sistema bibliotecario. Non descrive lo stato completo del sistema in nessuna bolla, né tutte le possibili transizioni di stato. Non è consigliabile modellare un sistema in un unico STD grande e monolitico. Una tale struttura

diventa presto ingombrante e difficile da capire. Sebbene potessimo modellare il sistema in una serie di FSM, avremmo comunque il problema di come integrarli in un modello.

Una possibile via d'uscita è consentire una scomposizione gerarchica degli FSM. Questa è l'essenza di una notazione nota come **diagrammi di stato.** Nelle carte di stato, i gruppi di stati possono essere visti come una singola entità a un livello, da perfezionare al livello successivo di astrazione. In UML, gli FSM sono modellati come diagrammi di macchine a stati (**Sezione 10.3.2**).

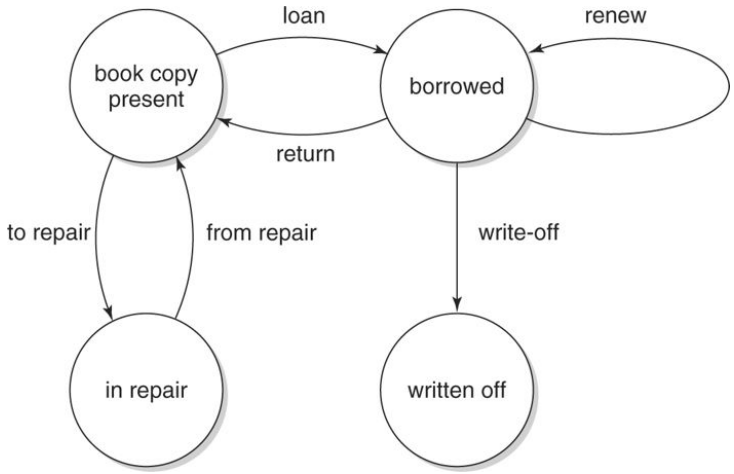


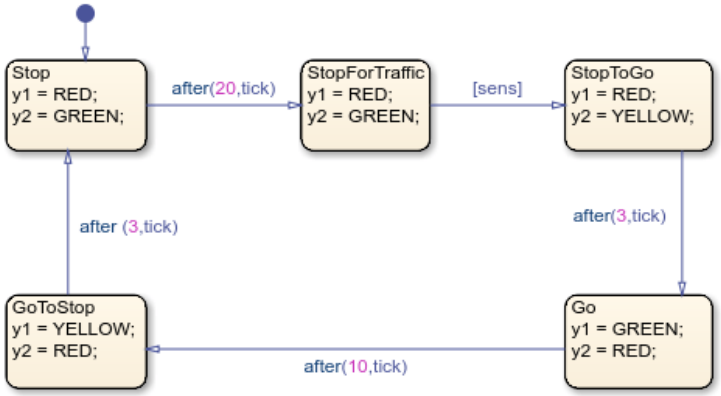
Figure 10.1 Un diagramma di transizione di stato

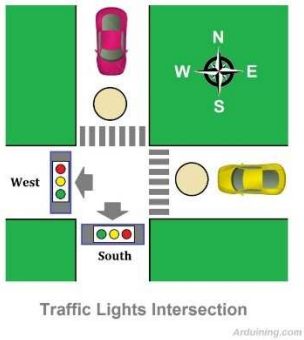
Esistono diversi **tipi principali di FSM.** In comune, i diversi tipi di FSM hanno un insieme finito di stati, un insieme di transizioni tra stati, un insieme (anch'esso in genere finito) di input e di output. Si distinguono per **come producono l'output** e **come gestiscono gli stati**. Ecco le principali categorie:

1. **Macchine a stati finiti di Moore**

- **Definizione:** l'output dipende **solo dallo stato corrente**.
- Ogni stato ha associato un valore di output fisso.
- **Transizioni:** determinate solo dall'input, ma l'output cambia solo quando cambia lo stato.

Questo esempio presso dalla documentazione di MatLab mostra come utilizzare la semantica di Moore per modellare un semaforo. Il modello del semaforo contiene una macchina di Moore denominata Light_Controller, che opera in cinque stati del traffico. Ogni stato rappresenta il colore del semaforo in due direzioni opposte, Nord-Sud ed Est-Ovest, e la durata del colore corrente. Il nome di ogni stato rappresenta il funzionamento della luce vista dalla direzione nord-sud.



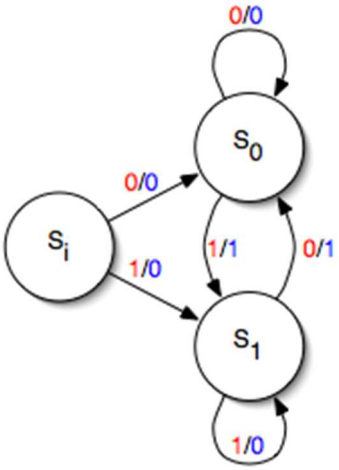


Questo modello utilizza la logica temporale per regolare le transizioni di stato. L'operatore **after** implementa un timer per il conto alla rovescia, che viene inizializzato quando si entra nello stato. Per impostazione predefinita, il timer fornisce un semaforo verde più lungo in direzione est-ovest rispetto alla direzione nord-sud perché il volume del traffico è maggiore sulla strada est-ovest. La luce verde in direzione Est-Ovest rimane accesa per almeno 20 ticchettii, ma può rimanere verde finché non arriva traffico in direzione Nord-Sud. Un sensore rileva se le auto sono in attesa al semaforo rosso in direzione nord-sud. In tal caso, il semaforo diventa verde in direzione nord-sud per mantenere il traffico in movimento. Quindi gli input sono il timer rappresentato dall'operator **after** e il sensore rappresentato da **[sens]**.
Gli **output** della macchina sono y1 e y2 per le luci rispettivamente Nord-Sud e Est-Ovest (l'output è espresso con un assegnamento).
Il Light_Controller si comporta come una macchina di Moore perché aggiorna i suoi output in base allo stato corrente prima di passare a un nuovo stato.
A esempio nello stato Stop, il semaforo è rosso per Nord-Sud, verde per Est-Ovest. Dopo 20 tick di clock, lo stato attivo diventa StopForTraffic in cui controlla il sensore – ma la luci non cambiano. Se il sensore indica che le auto sono in attesa ([sens] è true) in direzione nord-sud, lo stato attivo diventa StopToGo e la luce y2 diventa YELLOW.

Vantaggi: più semplice da analizzare e meno soggetta a glitch (l'output cambia solo su cambio di stato).
Svantaggi: meno reattiva, perché l'output può aggiornarsi solo al passaggio di stato.

2. Macchina di Mealy

- **Definizione:** l'output dipende **sia dallo stato corrente** che **dall'input**.
- L'output può cambiare **immediatamente** quando cambia l'input, anche senza cambiare stato.



In questo esempio, una semplice macchina Mealy ha un ingresso (che può essere o 0 o 1) e un'uscita (che può essere o 0 o 1). Ogni transizione è etichettata con il valore dell'input (mostrato in rosso) e il valore dell'output (mostrato in blu). La macchina si avvia nello stato *Si*., l'output è l'OR ESCLUSIVO (XOR) dei due valori di input più recenti; quindi, la macchina implementa un rilevatore di "rampa", emettendo un 1 ogni volta che l'input si capovolge e uno 0 in caso contrario.

Ad esempio, se in Si riceve uno 0, va in S_0 e produce 0.

Vantaggi: più compatta (meno stati), più reattiva.
Svantaggi: può essere più complessa da progettare e soggetta a glitch se gli input cambiano rapidamente.

3. **Macchine Deterministiche e Non Deterministiche**
   - **Deterministiche (DFA):** per ogni stato e input esiste **una sola** transizione possibile.
   - **Non Deterministiche (NFA):** per un dato stato e input possono esistere **più** transizioni possibili (o nessuna).
     Gli NFA sono usati soprattutto in teoria dei linguaggi formali (automata theory), non tanto nei sistemi reali.
4. **FSM con Output o senza Output**
   - **FSM di riconoscimento (acceptor):** usate per riconoscere linguaggi, producono solo un risultato "accetta/rifiuta".
   - **FSM di controllo o generazione:** producono una sequenza di output in funzione degli input e degli stati.
5. **FSM Estese (EFSM – Extended Finite State Machine)**
   - Variante delle FSM classiche in cui ogni transizione può avere **condizioni (guardie)** e **azioni** basate su **variabili interne**.

- Possono gestire più informazione di una semplice FSM, quindi sono più espressive.

Usate in: ingegneria del software, modellazione UML (diagrammi di stato), protocolli di comunicazione, ecc.

10.1.3 Diagrammi di flusso dei dati

Il metodo di progettazione del flusso di dati è nato nei primi anni '70 con Yourdon e Constantine. Nella sua forma più semplice, la progettazione del flusso di dati non è che una scomposizione funzionale rispetto al flusso di dati. Un componente (modulo) è una scatola nera che trasforma un flusso di input in un flusso di output. La notazione principale utilizzata è quella dei diagrammi di flusso dei dati (DFD).

In un diagramma di flusso di dati si distinguono quattro tipi di entità di dati:

- **Le entità esterne** sono l'origine o la destinazione di una transazione. Queste entità si trovano al di fuori del dominio considerato nel diagramma di flusso dei dati. Le entità esterne sono indicate come quadrati.
- **I processi** trasformano i dati. I processi sono indicati da cerchi.
- **Gli archivi dati** si trovano tra due processi e sono luoghi in cui le strutture di dati vengono archiviate fino a quando non sono necessarie. Sono indicati dal nome dell'archivio dati tra due linee parallele.
- **I flussi di dati** tra processi, entità esterne e archivi di dati sono percorsi lungo i quali viaggiano le strutture di dati. Un flusso di dati è indicato da una freccia.

I diagrammi di flusso dei dati risultano da un processo di scomposizione dall'alto verso il basso. Il diagramma al livello più alto ha un solo processo, che denota 'il sistema'. Questo diagramma di primo livello viene quindi ulteriormente scomposto. Per il nostro esempio di libreria, questo potrebbe portare al diagramma del flusso di dati della **Figura 10.3** . Una richiesta del cliente viene prima analizzata in un processo denominato "elaborazione preliminare". Di conseguenza, viene attivato uno tra 'titolo in prestito' o 'titolo di ritorno'. Entrambi questi processi aggiornano un archivio dati denominato "amministrazione del catalogo". Le richieste dei client vengono registrate in un "file di registro" dell'archivio dati. Questo archivio dati viene utilizzato per produrre report di gestione.

Il metodo di progettazione maggiormente utilizzato insieme ai diagrammi di flusso dei dati è discusso nella **Sezione 12.2.2** . Lì, forniamo anche altri diagrammi di flusso di dati di esempio.

10.1.4 Carte CRC

CRC sta per Classe–Responsabilità–Collaboratori. Una scheda CRC è semplicemente una scheda 4"×6" o 5" × 7" con tre campi etichettati Classe, Responsabilità e Collaboratori. Le schede CRC sono state sviluppate in risposta all'esigenza di documentare le decisioni di progettazione collaborativa. Le schede CRC sono particolarmente utili nelle prime fasi dello

sviluppo del software, per aiutare a identificare i componenti, discutere problemi di progettazione in team multidisciplinari e specificare i componenti in modo informale. Le carte CRC possono essere definite uno strumento a bassa tecnologia, al contrario degli strumenti ad alta tecnologia che utilizziamo comunemente. Eppure sono molto utili. È anche divertente lavorare con loro nei nostri incontri di lavoro troppo seri.

Le schede CRC non vengono utilizzate solo nelle sessioni di progettazione collaborativa. All'interno della comunità dei modelli di progettazione, ad esempio, vengono utilizzati per documentare gli elementi che partecipano a un modello.

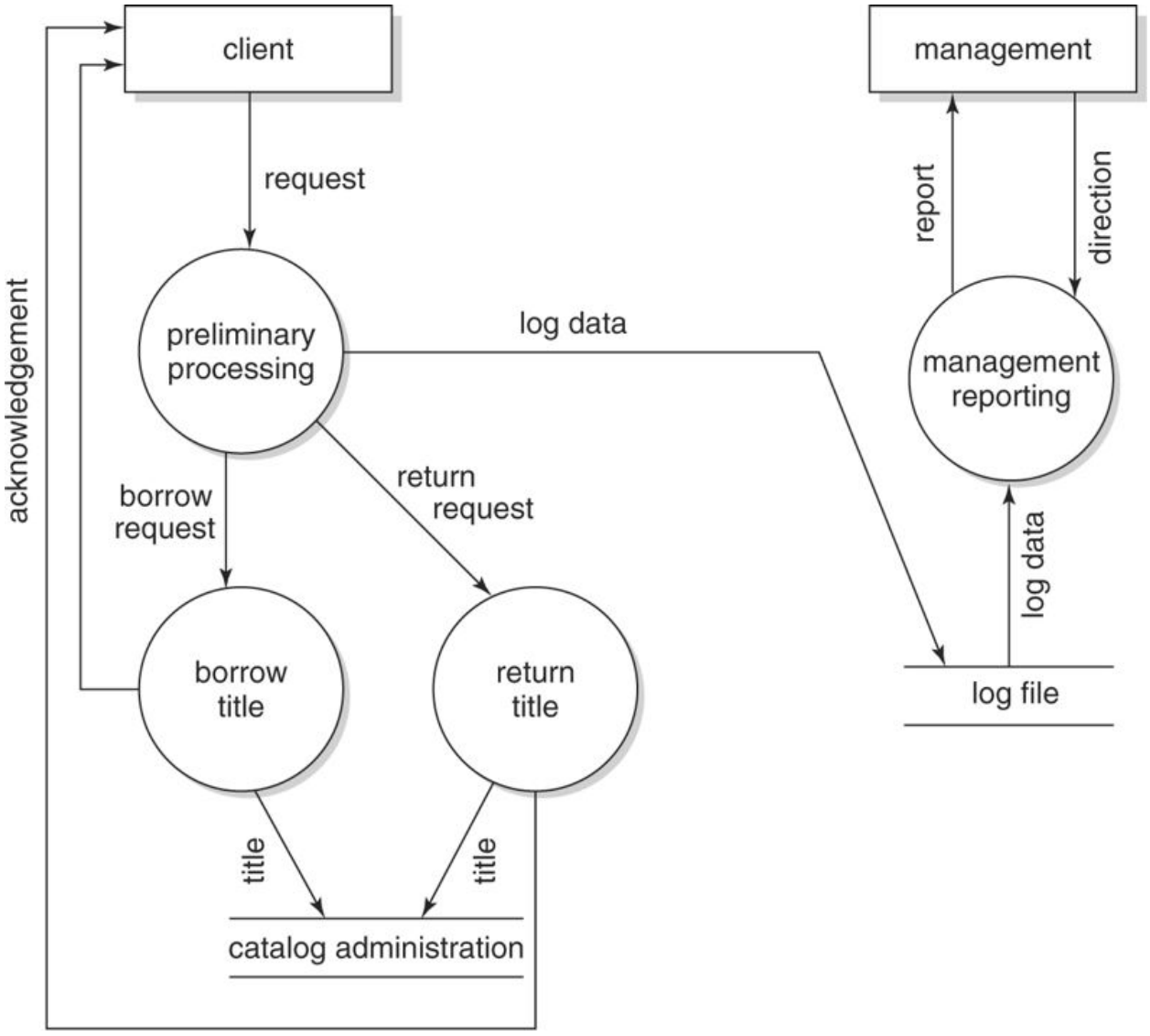


Figura 10.3 Diagramma del flusso di dati per l'automazione della libreria

La parola "classe" in CRC è una reliquia storica. Le carte CRC possono essere utilizzate per descrivere qualsiasi elemento di design. Tuttavia, ci atterremo alla terminologia originale. Il nome della classe appare nell'angolo in alto a sinistra della scheda. Sotto il nome della classe

compare un elenco puntato di responsabilità e nella parte destra della scheda compare un elenco di collaboratori. La Figura 10.4 fornisce un esempio di una tessera CRC per un componente Prenotazioni nel nostro sistema bibliotecario.

Le principali responsabilità di questa componente sono di mantenere un elenco aggiornato delle prenotazioni e di gestire le prenotazioni su base FIFO. I suoi collaboratori sono i componenti Catalogo e Sessione Utente. I tipi di interazione con questi componenti sono mostrati nella Figura 10.13 .

10.1.5 Linguaggi formali

Vedi i lucidi

10.1.6 Notazioni operative e notazioni dichiarative

Vedi lucidi

10.2 MODELLAZIONE AD OGGETTI

- Ciò che conta non è quanto da vicino modelliamo la realtà odierna, ma quanto sia estensibile e riutilizzabile il nostro software.
  Meyer (**1996**)

Il mondo che ci circonda è pieno di oggetti, animati e inanimati, concreti e astratti: alberi e tavoli, automobili e cause legali. Secondo alcuni, l'analisi e il design riguardano la modellazione di quegli oggetti del mondo reale. In generale, questa visione ha le sue origini nella scuola scandinava di progettazione del linguaggio di programmazione (SIMULA-67) e sviluppo di software. Può essere definita la visione europea. Secondo altri, l'analisi e la progettazione riguardano l'identificazione dei componenti riutilizzabili e la costruzione della loro gerarchia di eredità. Quest'ultimo punto di vista, che può essere definito il punto di vista americano, si mostra chiaramente nella citazione di cui sopra.

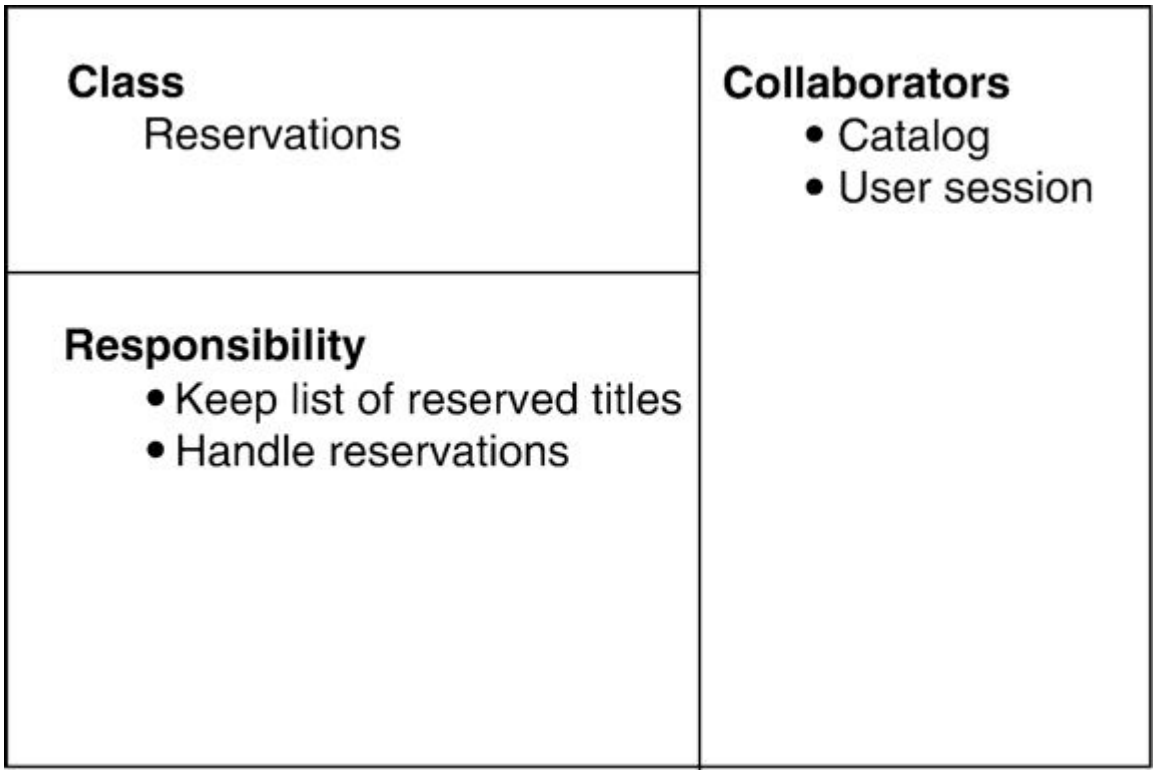


Figura 10.4 Una carta CRC

cos'è allora un oggetto? Come ci si potrebbe aspettare, ci sono punti di vista diversi su ciò che la nozione di oggetto comporta. Possiamo distinguere i seguenti punti di vista:

- Il punto di vista della modellazione: un oggetto è un modello concettuale di una parte di un mondo reale o immaginario. Da questo punto di vista, caratteristiche importanti sono:
  - – ogni oggetto ha un'identità, che lo distingue da tutti gli altri oggetti;
  - – gli oggetti hanno sostanza: proprietà che mantengono e possono essere scoperte investigando un oggetto.

Da un punto di vista pratico, l'identità dell'oggetto è un tag immutabile, o indirizzo, che identifica in modo univoco quell'oggetto. Oggetti diversi occupano diverse regioni della memoria. Pragmaticamente, inoltre, gli oggetti possono essere considerati come implementazioni di tipi di dati astratti (ADT). Un oggetto è quindi costituito da uno stato mutabile, cioè l'insieme di variabili dell'ADT, e da operazioni per modificare o ispezionare lo stato. In genere, l'unico modo per accedere a un oggetto è tramite queste operazioni. Le operazioni fungono quindi da interfaccia all'oggetto. Un oggetto è quindi un insieme di tre aspetti:

- `oggetto = identità + variabili + operazioni`

o oppure

- `oggetto = identità + stato + comportamento`

- Il punto di vista filosofico: gli oggetti sono astrazioni esistenziali, in opposizione alle astrazioni universali. In alcuni ambienti (in particolare il mondo Smalltalk), "tutto è un oggetto". In questa prospettiva, gli oggetti agiscono come una nozione unificante alla base di tutto il calcolo. Tuttavia, si può anche sostenere che esistono due tipi di astrazione piuttosto distinti. Alcuni tipi di entità hanno un inizio e una fine naturali. Vengono creati ad un certo punto nel tempo, esistono per un po' e vengono distrutti. I tipi di entità modellati come oggetti durante l'analisi e la progettazione appartengono in genere a questa classe. Altri tipi di entità, come numeri, date e colori, hanno un'esistenza "eterna". Non sono istanziati; non possono essere modificati; loro 'vivono' per sempre. Queste entità sono generalmente denominate valori.
- Il punto di vista dell'ingegneria del software: gli oggetti sono astrazioni di dati, dati incapsulanti e operazioni su quei dati. Questo punto di vista sottolinea la località dell'informazione e l'indipendenza della rappresentazione; si vedano anche **le Sezioni 12.1.1** e **12.1.3** . Tuttavia, non tutti i linguaggi di programmazione impongono l'astrazione dei dati e gli oggetti non devono sempre incapsulare un tipo di dati astratto. Potremmo affermare che l'astrazione dei dati e gli oggetti sono dimensioni in qualche modo ortogonali e indipendenti.

Un linguaggio di programmazione che ci consente semplicemente di incapsulare tipi di dati astratti nei moduli è spesso definito **basato su oggetti.** L'aggettivo "orientato agli oggetti", quindi, è riservato alle lingue che supportano anche L'**ereditarietà** (discusso di seguito).

- Il punto di vista dell'implementazione: un oggetto è una struttura contigua in memoria. Tecnicamente, un oggetto può essere considerato come un record di dati ed elementi di codice. Un oggetto può essere composto o aggregato (che possiede altri oggetti). I sottooggetti possono, a loro volta, possedere sottooggetti anche "più piccoli", e così via. Gli oggetti di livello più basso in questa gerarchia sono oggetti atomici, che in genere denotano numeri interi, numeri reali o booleani.

L'attuazione di questa relazione "posseduti da" sembra essere intricata. Da un lato, gli oggetti possono essere contenuti in altri oggetti. In questa rappresentazione, tutti i riferimenti sono eliminati. Non esiste il concetto di condivisione. Questo schema è noto come **semantica dei valori.** La semantica del valore è inadeguata per i sistemi orientati agli oggetti, poiché tali sistemi richiedono oggetti condivisibili. Lo schema opposto è la **semantica di riferimento:** i dati sono rappresentati come un oggetto atomico o come un aggregato di riferimenti ad altri oggetti. La semantica di riferimento pura è inefficiente nel caso di oggetti primitivi come numeri interi o caratteri. Come schema di archiviazione viene comunemente utilizzata una combinazione in cui gli oggetti aggregati possono contenere altri oggetti, fare riferimento ad altri oggetti o eseguire entrambe le cose contemporaneamente. La scelta di un particolare modello di archiviazione si riflette, in una certa misura, nella semantica del linguaggio di alto livello (ad esempio, quando si tratta di copiare o confrontare oggetti).

- Il punto di vista formale: un oggetto è una macchina a stati con un insieme finito di stati e un insieme finito di funzioni di stato. Queste funzioni di stato mappano i vecchi stati e ingressi a nuovi stati e uscite.
- La formalizzazione dei concetti e delle costruzioni dei linguaggi orientati agli oggetti è difficile. I formalismi matematici tendono ad essere basati sui valori. I concetti imperativi, come lo stato e la condivisione, che sono centrali nei linguaggi orientati agli oggetti non si adattano facilmente all'interno di tali schemi.

Durante l'analisi, il punto di vista concettuale è solitamente enfatizzato. Coloro che sono dell'opinione che l'analisi e il design si fondono armoniosamente l'uno nell'altro tendono a mantenere questo punto di vista durante la progettazione. Altri, invece, sono dell'opinione che l'analisi e il design siano diversi, indipendentemente dal fatto che siano o meno orientati agli oggetti. È probabile che sottolineino altri punti di vista durante la progettazione. La definizione di un oggetto data in (Coad e Yourdon, **1991**) riflette anche la tensione tra un punto di vista orientato al problema e uno orientato alla soluzione: un oggetto è " *un'astrazione* di qualcosa in un dominio problematico, che riflette le capacità di un sistema conservare informazioni su di esso, interagire con esso o entrambi; un *incapsulamento* dei valori di Attributo e dei loro Servizi esclusivi'. Torneremo su questa dicotomia quando parleremo di metodi orientati agli oggetti nella **Sezione 12.3** .

Come notato sopra, gli oggetti sono caratterizzati da un insieme di attributi (proprietà). Un tavolo ha gambe, un piano del tavolo, dimensioni, colore, ecc. Il concetto di **attributo ha origine dalla modellazione entità-relazione;** vedere **la Sezione 10.1.1** . In ERM, gli attributi rappresentano *proprietà intrinseche* delle entità, proprietà il cui valore non dipende da altre entità. Gli attributi denotano proprietà identificative e descrittive, come nome o peso. *Le relazioni* d'altra parte denotano proprietà *reciproche,* come un dipendente assegnato a un progetto o un libro preso in prestito da un membro. In UML, queste relazioni sono chiamate associazioni. In UML, la distinzione tra attributi e relazioni formalmente non esiste. Entrambi sono proprietà di una classe. È considerata buona pratica in UML modellare proprietà semplici come attributi e proprietà più complesse come associazioni.

Nel contesto della modellazione orientata agli oggetti, il termine attributo viene talvolta utilizzato per denotare qualsiasi campo nella struttura dei dati sottostante. In tal caso, l'identità dell'oggetto è un attributo, lo stato denota l'insieme di attributi "strutturali" e le operazioni denotano gli attributi "comportamentali". Usiamo il termine 'attributo' per denotare un attributo strutturale. Collettivamente, l'insieme degli attributi di un oggetto costituisce quindi il suo stato. Include le proprietà intrinseche, solitamente rappresentate come valori, così come le proprietà reciproche, solitamente rappresentate come riferimenti ad altri oggetti.

A livello di linguaggio di programmazione, si dice che gli oggetti che hanno lo stesso insieme di attributi appartengano alla stessa *classe.* I singoli oggetti di una classe sono chiamati *istanze*

di quella classe. Quindi potremmo avere una tabella di classe, con le istanze MyTable e YourTable. Queste istanze hanno gli stessi attributi, con valori possibilmente diversi.

Un oggetto non solo racchiude il suo *stato,* ma anche il suo *comportamento,* cioè il modo in cui agisce su altri oggetti ed è agito da altri oggetti. Il comportamento di un oggetto è descritto in termini di *servizi* forniti da quell'oggetto. Questi servizi vengono invocati *inviando messaggi* dall'oggetto che richiede il servizio all'oggetto su cui si agisce.

Affinché una raccolta di oggetti funzioni insieme come previsto, ciascuno degli oggetti deve poter fare affidamento sul corretto funzionamento degli oggetti con cui interagisce. In una vista *client-server,* un oggetto, il client, richiede un servizio da un altro oggetto, il server. Questa dipendenza reciproca può essere vista come un contratto tra gli oggetti coinvolti. Il cliente non chiederà più di quanto indicato nel contratto, mentre il server si impegna a consegnare quanto stabilito nel contratto. In questa prospettiva, i servizi sono indicati anche come *responsabilità.*

Il principale aspetto comportamentale di un oggetto riguarda i cambiamenti di stato. Lo stato di un'istanza dell'oggetto non è statico, ma cambia nel tempo: l'istanza dell'oggetto viene creata, aggiornata e infine distrutta. Inoltre, alcune informazioni possono essere richieste da un oggetto. Queste informazioni possono riguardare lo stato dell'istanza dell'oggetto, ma possono anche implicare un calcolo di qualche tipo.

Ad esempio, un cliente di una libreria può avere attributi come Nome, Indirizzo e BooksOnLoan. Deve essere possibile creare un'istanza del tipo di oggetto Cliente. In tal caso, devono essere forniti valori adeguati per i suoi attributi. Una volta creata l'istanza, sono possibili cambiamenti di stato: i libri vengono prestati e restituiti, il cliente cambia indirizzo, ecc. Infine, l'istanza viene distrutta quando il cliente cessa di essere membro. Le informazioni richieste possono riguardare cose come un elenco di libri in prestito o il numero di libri in prestito. Il primo fa parte dello stato che descrive un particolare cliente e può essere recuperato direttamente da quello stato. NumberOfBooksOnLoan è un servizio che richiede un calcolo di qualche tipo, ad esempio il conteggio del numero di elementi in BooksOnLoan.

In genere non ci occupiamo di singoli oggetti. Il nostro obiettivo è identificare e mettere in relazione i tipi di oggetti (cioè le classi). Useremo spesso il termine oggetto per denotare un tipo di oggetto. Una delle nostre principali preoccupazioni durante l'analisi e la progettazione è identificare questo insieme di oggetti, insieme ai loro attributi (stato) e servizi (comportamento).

Le relazioni tra oggetti possono essere espresse in una struttura di classificazione. I principali tipi di relazione rappresentati in tali strutture sono elencati nella **Tabella 10.2** . Se abbiamo oggetti Tavolo e Sedia, possiamo anche definire un oggetto più generale Arredamento. Si dice che Tavolo e Sedia siano *specializzazioni* di Mobili, mentre Mobili è una *generalizzazione* di

Tavolo e Sedia. Queste relazioni sono anche conosciute come relazioni "è-a". La relazione è-una è un concetto ben noto della modellazione entità-relazione.

Tabella 10.2 Principali tipi di relazioni tra oggetti

Relazione	Esempio
Specializzazione-generalizzazione, is-a	Il tavolo *è un* mobile
Intera parte, ha	Il tavolo *ha* TableTop
Membro di, ha	La biblioteca *ha un* membro

Le relazioni specializzazione-generalizzazione possono essere espresse in una struttura gerarchica come quella della **Figura 10.5** . Nella sua forma più generale, la struttura di classificazione è un grafo aciclico diretto (DAG). Molte strutture di classificazione possono essere rappresentate come un albero, in cui ogni oggetto è un discendente diretto esattamente di un altro oggetto. A livello di linguaggio di programmazione, L'**ereditarietà singola** corrisponde a una struttura ad albero, mentre **l'ereditarietà multipla** corrisponde a un DAG.

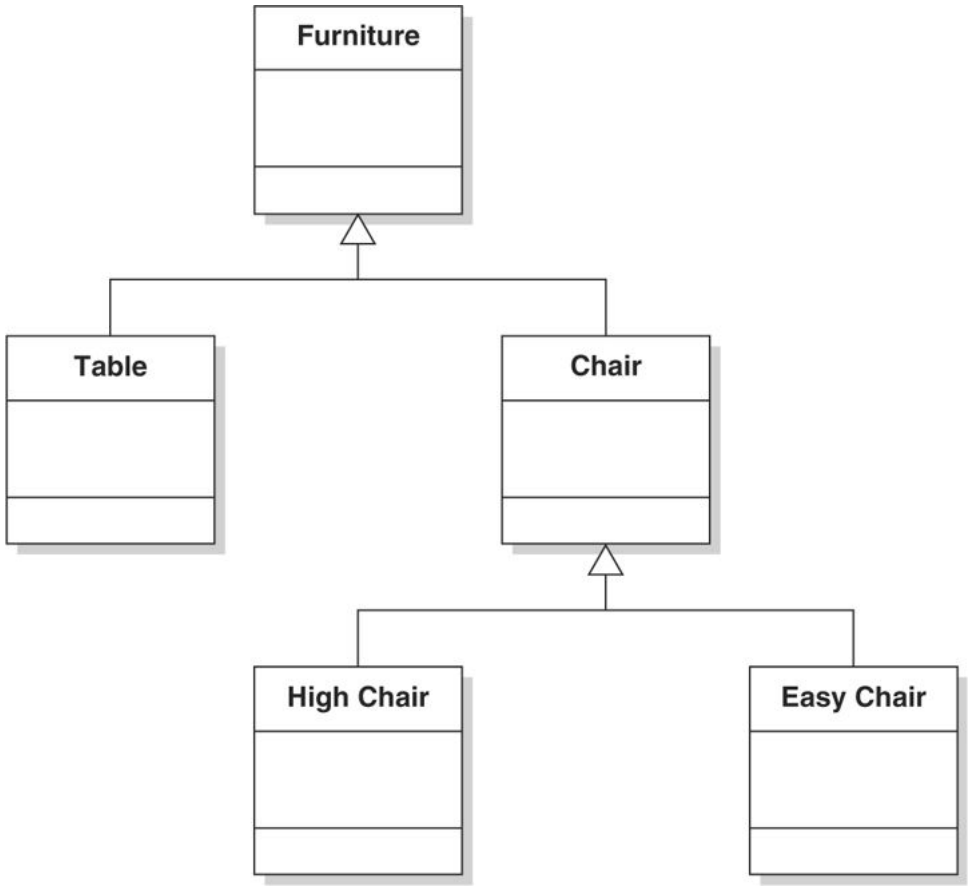


Figura 10.5 Gerarchia degli oggetti

Oggetti diversi possono condividere alcuni attributi. Sia i tavoli che le sedie hanno un'altezza, per esempio. Anziché definire l'insieme completo di attributi per ciascun oggetto, possiamo definire attributi comuni a un livello superiore nella gerarchia degli oggetti e lasciare che i discendenti *ereditino* tali attributi. Possiamo quindi definire l'attributo Altezza a livello di Mobili piuttosto che a livello di ciascuno dei suoi discendenti. Ovviamente, questo è solo un altro modo di guardare alla gerarchia degli oggetti. Il fatto che Chair e Table siano entrambi discendenti di Furniture suggerisce già che condividano alcune proprietà, proprietà comuni ai vari tipi di mobili. Il fatto che siano diversi discendenti di Furniture suggerisce anche che ognuno di essi abbia proprietà uniche.

In alternativa, possiamo vedere la gerarchia di oggetti come una *gerarchia di tipi.* Sedia e Tavolo sono *sottotipi* di Mobili, allo stesso modo in cui Cardinal è un sottotipo di Integer. In questa prospettiva, un oggetto è una restrizione degli oggetti di cui è una specializzazione. Ogni sedia è un mobile ma non è vero il contrario.

Collegando esplicitamente gli oggetti nella gerarchia degli oggetti, si realizza un legame semantico molto più stretto tra oggetti correlati di quanto sia possibile in approcci di progettazione più tradizionali. In una scomposizione funzionale del nostro problema di automazione delle biblioteche, ad esempio, non c'è praticamente alcun modo per rendere esplicite nella progettazione le somiglianze tra libri e riviste. In un design orientato agli oggetti, gli oggetti Libro e Rivista possono essere resi discendenti di un oggetto più generale. Pubblicazione e attributi come Editore possono essere ereditati da questo tipo più generale di oggetto.

La relazione is-a è un modo per organizzare i tipi (oggetto) in una gerarchia. La parte di relazione è un'altra importante proprietà organizzativa dei tipi di oggetto. Un tavolo "ha" un piano e gambe. Una pubblicazione "ha" una descrizione del titolo e un editore. Questa parte della relazione *aggrega* i componenti in un "tutto". Descrive come le cose composte siano composte da cose più semplici. Per definizione, il composto è a un livello di astrazione più elevato rispetto ai suoi componenti.

Un oggetto come TableTop è composto da attributi, ad esempio Colore, Larghezza e Lunghezza. Al livello successivo, oggetti come TableTop e Legs vengono assemblati in un oggetto di livello superiore, vale a dire. Tavolo. A quel livello, possiamo introdurre attributi aggiuntivi, come Size, in modo che anche la tabella possa essere vista come un aggregato del primo tipo. In generale, un oggetto composto è costituito da un numero di (riferimenti a) altri oggetti e da un numero di attributi 'semplici', cioè valori.

Nel caso di Table, la parte della relazione è una parte della relazione del mondo reale. Nel caso della pubblicazione, l'editore non corrisponde a una parte dell'oggetto del mondo reale sottostante. Fa semplicemente parte della *rappresentazione* dell'oggetto Pubblicazione. A

volte, viene fatta una distinzione esplicita tra la parte della relazione del mondo reale e la parte rappresentativa della (o componente della) relazione. In UML, sono chiamati rispettivamente *composizione* e *aggregazione.*

In molti metodi di modellazione, la parte di relazione sussume la *parte di* relazione. La relazione membro-di viene utilizzata per modellare la relazione tra un insieme e i suoi membri. Tuttavia, a volte è utile essere in grado di distinguere tra queste proprietà organizzative. Ad esempio, la parte della relazione è generalmente considerata transitiva, mentre la parte della relazione non lo è. Se Book è membro di Library e Library è membro di PublicInstitutions, non vogliamo dedurre che Book sia membro di PublicInstitutions.

10.3 UML: IL LINGUAGGIO UNIFICATO DI MODELLAZIONE

Il linguaggio di modellazione unificato ha le sue radici nell'analisi orientata agli oggetti e nei metodi di progettazione degli anni '80. Diversi attori chiave in questo campo (Grady Booch, John Rumbaugh e Ivar Jacobson) si sono uniti a Rational e hanno iniziato a unificare i loro metodi e notazioni. Ciò ha portato alle prime versioni di UML. In una fase successiva, è subentrato l'Object Management Group (OMG), un consorzio aperto di aziende. OMG ora controlla le attività intorno a UML e lo ha adottato come uno dei suoi standard. UML è di gran lunga la notazione più utilizzata sia per l'ingegneria dei requisiti che per la progettazione. La versione attuale è nota come UML 2.

I 13 diagrammi di UML 2 sono elencati nella **Tabella 10.3** . Alcuni diagrammi, come il diagramma delle classi e il diagramma della macchina a stati, sono presenti dall'inizio dell'orientamento agli oggetti. Altri sono più recenti. Ad esempio, il diagramma della struttura composita e il diagramma temporale sono stati introdotti in UML 2. Il diagramma delle classi è il diagramma più utilizzato (75%), seguito dal diagramma dei casi d'uso e dal diagramma di comunicazione (50%) (Dobing e Parsons, **2006**). Le persone di solito non aderiscono rigorosamente alla semantica della lingua (Lange *et al.*, **2006**). Alcuni dei diagrammi danno una vista statica. Ad esempio, un diagramma di classe mostra la struttura statica di un sistema. Altri diagrammi forniscono una vista dinamica o comportamentale, ovvero mostrano cosa succede quando il sistema viene eseguito. Ad esempio, un diagramma di sequenza mostra quali messaggi vengono scambiati tra istanze di classi. Nella **Tabella 10.3** , i diagrammi statici sono contrassegnati con una S ei diagrammi dinamici sono contrassegnati con una D. Nelle seguenti sottosezioni, discutiamo i diagrammi UML più importanti.

Tabella 10.3 Tipi di diagramma UML 2

Diagramma	Descrizione
Attività (D)	Modellare i processi aziendali, il flusso di lavoro, la logica procedurale. Simile a un diagramma di flusso ma supporta il parallelismo, come in una rete di Petri
Classe (S)	Per modellare le classi, le loro caratteristiche e relazioni (vedi **Sezione 10.3.1**)
Comunicazione (D)	Per modellare il flusso di messaggi tra istanze di classi. Molto simile al diagramma di sequenza (vedi **Sezione 10.3.4**)
Componente (S)	Per modellare un insieme di componenti e le loro interrelazioni, attraverso interfacce (vedi **Sezione 10.3.5**)
Struttura composita (D)	Per modellare la struttura dinamica interna di una classe
Distribuzione (S)	Per modellare il layout fisico, ovvero l'assegnazione di elementi di sistema ad elementi hardware
Panoramica dell'interazione (S)	Combina diagrammi di attività e diagrammi di sequenza
Oggetto (S)	Modellare oggetti e le loro relazioni in un determinato momento; noto anche come diagramma di istanza
Pacchi)	Per modellare il raggruppamento di elementi in pacchetti
Sequenza (D)	Per modellare l'ordine in cui i messaggi vengono scambiati tra istanze di classi (vedi **Sezione 10.3.3**)
Macchina a stati (D)	Per modellare gli stati in cui può trovarsi un oggetto e la transizione tra stati (vedi **Sezione 10.3.2**)
Tempi (D)	Per modellare i cambiamenti di stato di un oggetto nel tempo
Caso d'uso (D)	Per modellare i casi d'uso (vedere **Sezione 10.3.6**)

10.3.1Il diagramma delle classi

I diagrammi di classe descrivono la struttura statica di un sistema. Un diagramma di classe è un grafico in cui i nodi sono oggetti (classi) e gli spigoli sono relazioni tra oggetti. Decorando i bordi si possono modellare molti tipi di relazione. Queste relazioni si dividono in due gruppi: *generalizzazioni* e *associazioni.*

L'esempio più comune di un diagramma delle classi di generalizzazione descrive la gerarchia sottoclasse-superclasse. **La Figura 10.5** è un esempio di tale diagramma di classe. Le classi sono indicate da rettangoli che hanno tre scomparti. Questi scomparti contengono, dall'alto verso il basso:

- il nome della classe,
- l'elenco degli attributi della classe, e
- l'elenco delle operazioni della classe.

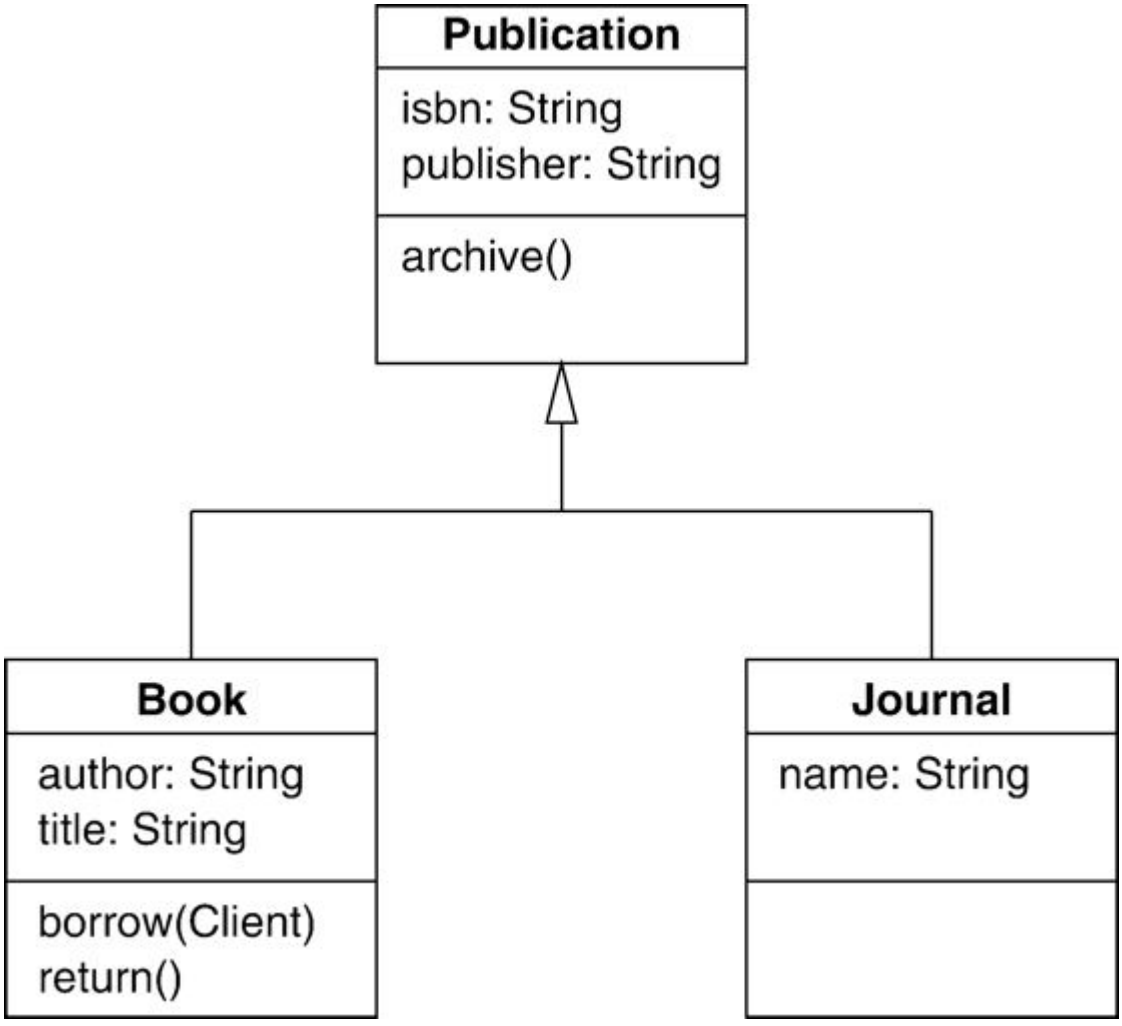


Figura 10.6 Diagramma delle classi UML: generalizzazione

UML consente una certa varietà nella sua notazione. Ad esempio, possiamo rappresentare una classe come un rettangolo con un solo compartimento, dando solo il nome della classe. Aggiungendo un po' più di dettaglio, possiamo rappresentare una classe come un rettangolo con due scomparti, dove il secondo caratterizza le responsabilità di quella classe, cioè cosa dovrebbe fare, come una specie di commento in linea. La **Figura 10.6** fornisce la rappresentazione a tre compartimenti in cui sono stati aggiunti numerosi dettagli a livello di analisi. Potremmo anche estendere ulteriormente la notazione e aggiungere dettagli a livello di implementazione, ad esempio se gli attributi e le operazioni sono pubblici o privati. Possiamo pensare a queste diverse rappresentazioni come viste diverse dello stesso elemento del modello. Possiamo immaginare il supporto dello strumento che consente all'utente di passare da una rappresentazione all'altra, sopprimendo o aggiungendo dettagli in caso di necessità.

Il triangolo cavo nella **Figura 10.6** indica che la struttura è una struttura di specializzazione-generalizzazione. La generalizzazione viene mostrata come un percorso solido dall'elemento più specifico (come Libro) all'elemento più generale (Pubblicazione), con un grande triangolo vuoto alla fine del percorso. Un gruppo di percorsi di generalizzazione può essere mostrato come un albero con un segmento condiviso, come in **Figura 10.6** .

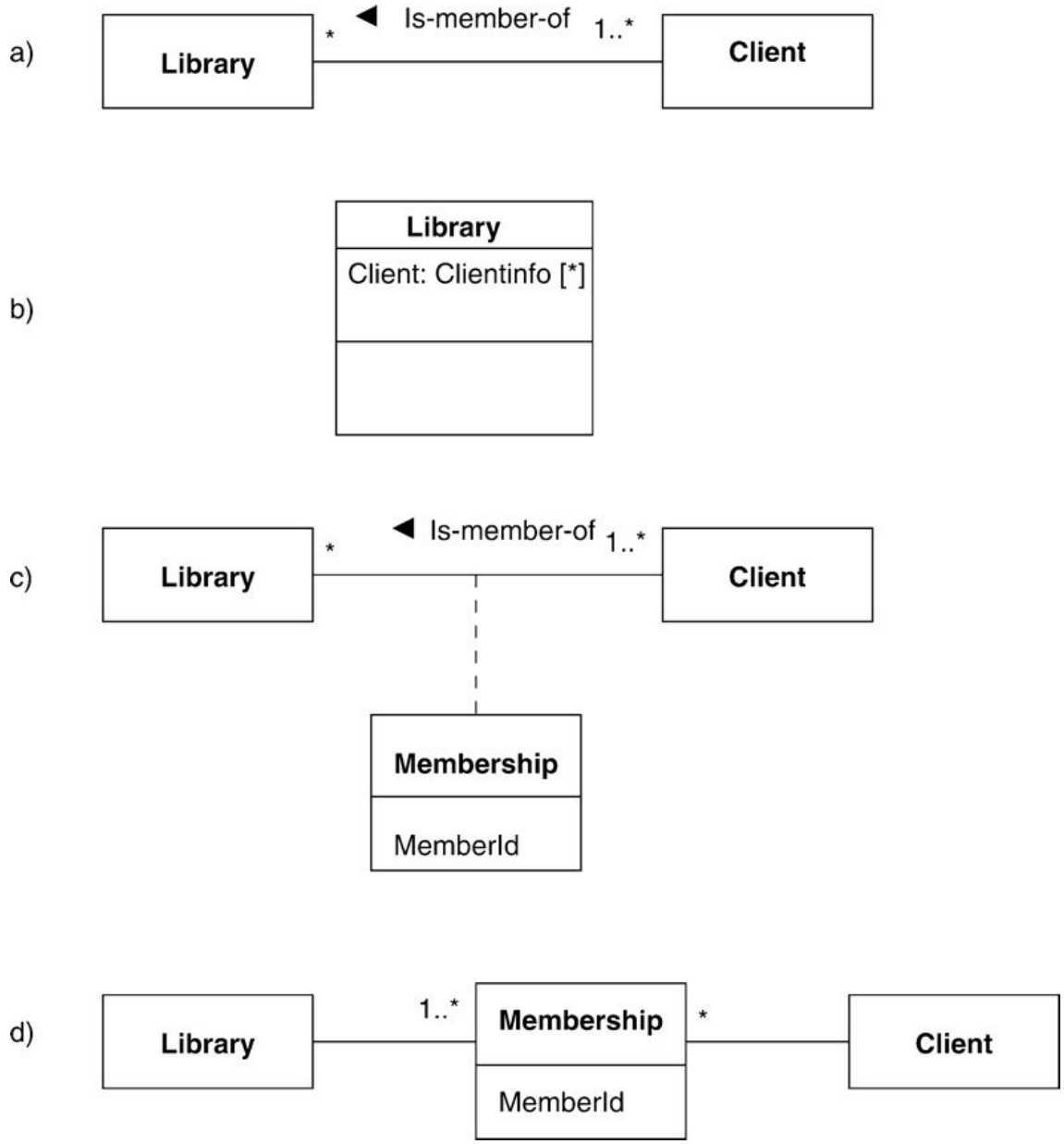


Figura 10.7 Diagramma delle classi UML: (a) associazione, (b) associazione come attributo, (c) classe di associazione, (d) classe di associazione come classe completa

Gli *attributi* di una classe denotano le *proprietà* di quella classe. Ad esempio, editore è una proprietà di Publication. Accanto agli attributi, UML ha un altro modo per denotare le proprietà di una classe, vale a dire. *associazioni.* Un'associazione UML è rappresentata come una linea continua che collega due classi. Questa linea può essere adornata con una varietà di glifi e informazioni testuali per fornire ulteriori dettagli della relazione. Una semplice associazione tra una libreria ei suoi client è illustrata nella **Figura 10.7a** . Accanto al percorso viene stampato il nome (facoltativo) dell'associazione. Il triangolo pieno indica la direzione in cui deve essere letto il verbo. Si noti che le associazioni sono bidirezionali: un cliente è

membro di una biblioteca e una biblioteca ha membri. Ulteriori ornamenti possono essere aggiunti per indicare le proprietà dell'associazione. Nella **Figura 10.7a** abbiamo aggiunto molteplici informazioni: un client può essere membro di una o più librerie, mentre una libreria può avere zero o più client.

A rigor di termini, non c'è differenza tra un attributo e un'associazione. Nella **Figura 10.7b** , abbiamo rappresentato il Cliente come un attributo della Libreria. Di solito, le proprietà semplici come numeri e date sono modellate come attributi, mentre i concetti più significativi sono modellati come associazioni.

Anche un'associazione come Member-of ha proprietà di classe. Ad esempio, questa associazione ha attributi, ad esempio MemberId, e operazioni, come BecomeMember e CeaseToBeMember. In alternativa, possiamo dire che l'appartenenza alla classe ha proprietà di associazione. In UML, questo elemento del modello è chiamato **classe di associazione.** Può essere rappresentato come un simbolo di classe attaccato da una linea tratteggiata a un percorso di associazione, come nella **Figura 10.7c** . Possiamo anche promuovere una classe di associazione a una classe completa, come nella **Figura 10.7d** . Nota che le molteplicità si sono spostate. Un'appartenenza (di un cliente) può riguardare una o più biblioteche, mentre l'appartenenza (della biblioteca) si riferisce a zero o più clienti.

La parte della relazione è chiamata **aggregazione** o **composizione** in UML. In un'aggregazione, gli oggetti possono far parte di più di un altro oggetto. Ad esempio, se la nostra biblioteca mantiene elenchi di letture obbligatorie per determinati corsi, un determinato libro potrebbe far parte di più di un elenco di letture obbligatorie. L'aggregazione è indicata da un diamante aperto come ornamento del ruolo dell'associazione. La composizione è una forte nozione di aggregazione, in cui l'oggetto parte può appartenere a un solo oggetto intero. Con la composizione, ci si aspetta che le parti vivano e muoiano con il tutto. Se un tavolo è composto da quattro gambe e un ripiano, il tavolo *possiede* queste parti. Non possono far parte di un altro tavolo contemporaneamente. La composizione è indicata da un diamante pieno pieno come ornamento del ruolo di associazione, come nella **Figura 10.8a** , che mostra un libro con il titolo delle parti, l'autore e l'isbn. Un libro ha un titolo e un ISBN, quindi queste parti hanno una molteplicità di 1. Assumiamo qui che un libro possa avere fino a tre autori, quindi quella parte ha una molteplicità di 1..3. Alla fine della composizione, la molteplicità è 1 o 0..1. Questa parte della relazione è una relazione tra una classe e le classi dei suoi attributi. Una notazione alternativa per questa parte di relazione consiste quindi nei primi due compartimenti del diagramma per una classe, come nella **Figura 10.8b** .

Oltre alla generalizzazione e all'associazione, ci sono molti altri modi in cui gli elementi di un diagramma di classe possono dipendere l'uno dall'altro. Ad esempio, una classe può chiamare operazioni da un'altra classe, creare istanze di un'altra classe e così via. Tali dipendenze sono rappresentate da una freccia tratteggiata, etichettata con il tipo di dipendenza. Se tutte le dipendenze sono incluse in un diagramma di classe, diventa presto molto disordinato. Quindi

è saggio includere solo le dipendenze importanti. Molti tipi di dipendenza non devono essere modellati manualmente, ma possono essere derivati dal codice sorgente ed esistono strumenti per farlo.

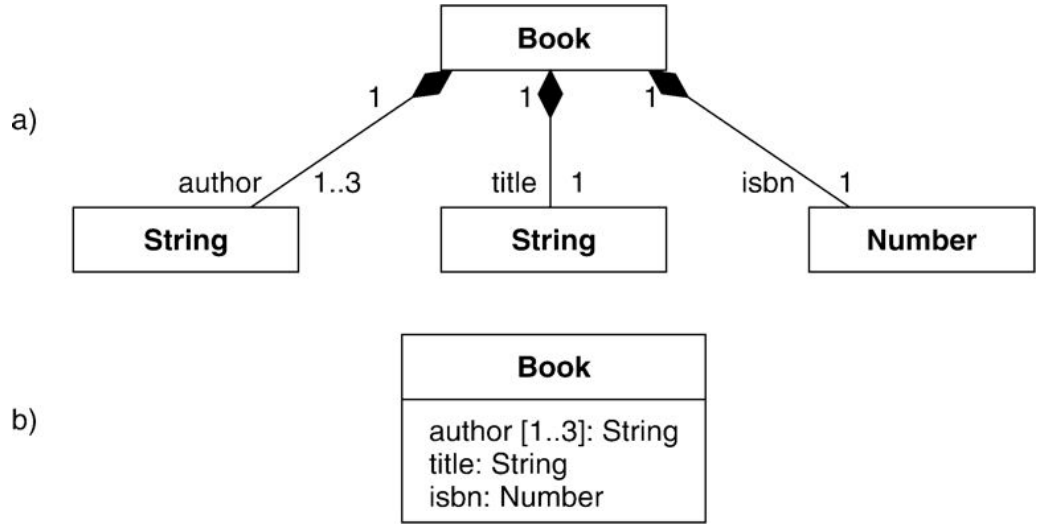


Figura 10.8 Diagramma delle classi UML: composizione come (a) ornamento del ruolo di associazione e (b) un semplice diagramma delle classi

Una *classe astratta* è una classe che non può essere istanziata direttamente. Solo i suoi clienti (concreti) possono farlo. Le classi astratte si verificano in genere in gerarchie di tipi di dati. Ad esempio, potremmo avere una lista di classi astratta, con sottoclassi come LinkedList e ArrayList. La classe astratta List può avere operazioni astratte, come get, che possono essere rese concrete solo a livello di sottoclasse. A livello di List, affermiamo semplicemente che ciascuna delle sue sottoclassi fornirà un'implementazione di get. Nel nostro esempio di libreria, avremmo potuto designare la pubblicazione come una classe astratta. Le classi astratte sono indicate stampando il loro nome in corsivo.

Un'interfaccia *è* una classe le cui caratteristiche sono tutte astratte. Non ha implementazione. Le interfacce sono un mezzo utile per suddividere l'insieme di proprietà di una classe in sottoinsiemi, nel caso in cui altre classi necessitino dell'accesso solo a sottoinsiemi di tali proprietà. Ad esempio, la classe Publication può avere proprietà accessibili ai clienti della libreria, nonché proprietà solo per uso interno, come il prezzo, chi ha autorizzato l'acquisizione e così via. Possiamo quindi definire due interfacce per la pubblicazione che sono rese disponibili ad altre classi nel sistema. La pubblicazione *fornisce quindi* interfacce diverse a classi client diverse, che a loro volta *richiedono* l'interfaccia. Le interfacce sono contrassegnate con la parola chiave «interfaccia», come in **Figura 10.9** . Le interfacce vengono spesso utilizzate per aumentare la robustezza di un modello, limitando l'accesso alle sole proprietà realmente necessarie.

10.3.2 Il diagramma della macchina a stati

Una delle principali classi di servizi forniti da un oggetto riguarda il ciclo di vita dell'oggetto: viene creata un'istanza dell'oggetto, aggiornata zero o più volte e infine distrutta. I diagrammi di transizione di stato, che descrivono i possibili stati di un oggetto e le transizioni tra questi stati, sono un buon aiuto nella modellazione di questo ciclo di vita.

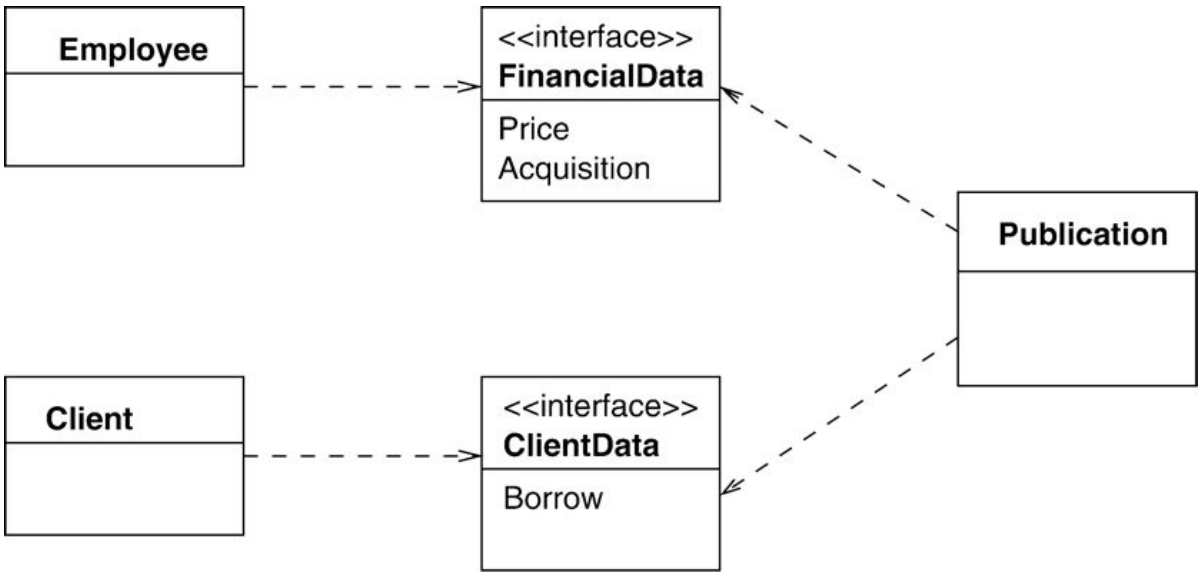


Figura 10.9 Diagramma delle classi UML: interfaccia

Solitamente, il modello della macchina a stati finiti e il diagramma di transizione di stato associato (vedi **Sezione 10.1.2**) sono estesi in diversi modi quando utilizzati nella modellazione del comportamento degli oggetti nel tempo:

- Nel classico modello di macchina a stati finiti, gli stati non hanno variabili locali. Tutte le informazioni necessarie sono codificate nello stato. Questo porta facilmente a modelli ingombranti. Ad esempio, supponiamo di voler modellare un oggetto LibraryMember come segue: una persona può diventare un membro della biblioteca, prendere in prestito fino a dieci libri e cessare di essere un membro della biblioteca. Questo porta a una macchina a stati finiti con stati come ha preso in prestito 0 libri, ha preso in prestito 1 libro, ha preso in prestito 2 libri,…, ha preso in prestito 10 libri. Se il numero di libri in prestito fosse modellato come una variabile locale, il numero di stati nel modello sarebbe ridotto da 12 a 2.

Per questo motivo, la macchina a stati finiti viene solitamente estesa aggiungendo variabili locali al modello. Uno stato in questa macchina a stati finiti estesa comprende quindi sia lo stato esplicito rappresentato da un nodo nel diagramma di transizione di stato sia il valore delle variabili del modello.

Queste variabili locali non vengono utilizzate solo per diminuire il numero di stati. Le transizioni di stato ora possono anche modificare i valori delle variabili; le variabili possono

essere testate per determinare lo stato successivo e le transizioni possono essere protette dal valore delle variabili. Nella **Figura 10.10** il numero dei libri in prestito è mantenuto nella variabile locale *N.* Questa variabile viene inizializzata a zero, aggiornata quando un libro viene preso in prestito o restituito e verificata quando una persona termina la sua iscrizione.

- I componenti modellati interagiscono con l'ambiente: ci sono eventi di input e azioni di output. In tutti i metodi di modellazione che conosciamo, gli eventi di input attivano le transizioni. Quando una persona diventa un membro della biblioteca, questo innesca la transizione iniziale; quando prende in prestito un libro, innesca una transizione da uno stato, diciamo, ha preso in prestito-7-libri a uno stato ha preso in prestito-8-libri. Se il modello ha variabili locali, quest'ultima transizione di stato può comportare una modifica del valore di tale variabile locale. Nella **Figura 10.10** , gli eventi di input sono indicati come stringhe che etichettano le transizioni di stato (come Start e Borrow).

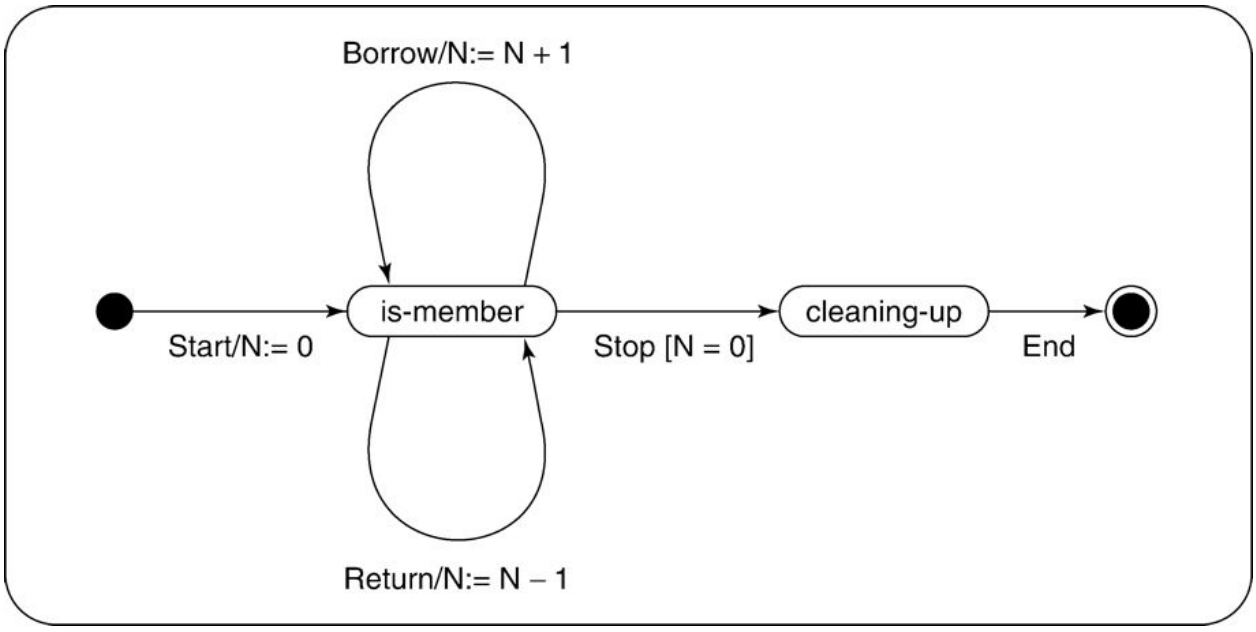


Figura 10.10 Diagramma macchina a stati UML: oggetto Membro

Diversi metodi di modellazione hanno modi diversi per gestire le azioni di output. A volte, le azioni di output sono associate a una transizione (questa è nota come macchina Mealy), a volte le azioni di output sono associate a uno stato (una macchina Moore). In quest'ultimo caso, l'azione di uscita viene eseguita non appena viene inserito lo stato. In senso formale, le macchine Mealy e le macchine Moore hanno la stessa potenza espressiva.

- I diagrammi a stati finiti possono diventare ingombranti. Pertanto, si può aggiungere una struttura, attraverso una gerarchia. Una parte del modello può essere compressa in uno stato. Se siamo interessati ai dettagli di uno stato, possiamo "ingrandire" quello stato.

Molti metodi di modellazione, incluso UML, descrivono la sequenza di stati che un oggetto attraversa in una variante dello **statechart.** I diagrammi di stato sono macchine a stati finiti

estese (cioè hanno variabili locali) in cui le azioni di output possono essere associate sia alle transizioni che agli stati e in cui gli stati possono essere organizzati gerarchicamente. In UML, questo tipo di diagramma è chiamato **diagramma della macchina a stati.** Come con i diagrammi di classe, UML ha una notazione ricca per i diagrammi di macchine a stati. Illustreremo gli ingredienti principali attraverso alcuni esempi (si vedano anche le **Figure 10.10** e **10.11**).

Uno stato è una condizione nella vita di un oggetto. Viene visualizzato come un rettangolo con angoli arrotondati. Uno stato iniziale (pseudo) viene mostrato come un piccolo cerchio pieno. Questo stato iniziale è un semplice dispositivo di notazione; un oggetto non può essere in tale stato. Indica il passaggio al primo stato "reale". Uno stato finale (pseudo) viene mostrato come un cerchio che circonda un piccolo cerchio pieno. Questo stato finale è anche un dispositivo di notazione. Una transizione viene mostrata come una freccia continua da uno stato all'altro. Quando si verifica un cambiamento di stato, quella transizione è detta "fuoco". Una transizione ha un'etichetta composta da tre parti. La forma generale è *trigger-firma [guardia]/attività.* Tutte e tre le parti sono opzionali. Il trigger-firma denota l'evento che attiva la transazione, come il prestito di un libro. L'evento può essere protetto da un'espressione booleana. Ad esempio, il passaggio dallo stato è membro alla ripulitura nella **Figura 10.10** è custodito dall'espressione ' *N* = 0': può verificarsi solo se il numero di libri in prestito è zero. Quando si verifica un evento, è possibile eseguire solo una transizione. Quindi, se si verificano più transizioni con lo stesso evento, le guardie devono escludersi a vicenda. L'etichetta di transizione può includere un'espressione procedurale dopo il simbolo '/'. Questa espressione procedurale viene eseguita quando si attiva la transizione.

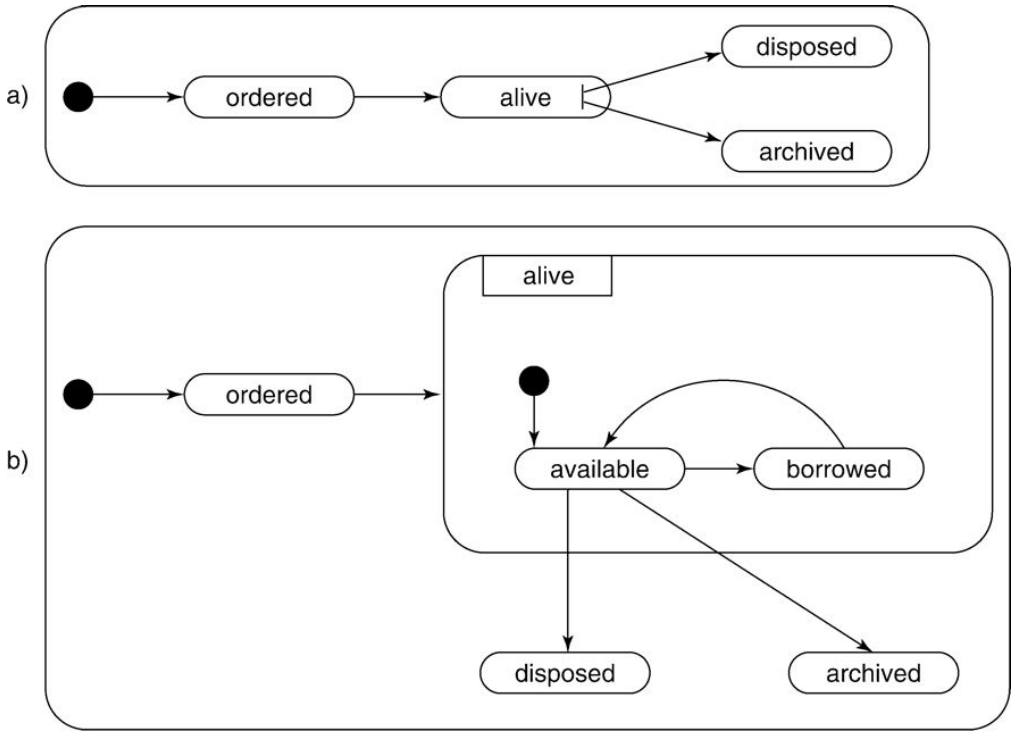


Figura 10.11 Diagramma della macchina a stati UML: oggetto Libro, (a) vista globale e (b) vista espansa

La Figura 10.11 fornisce un esempio di stati nidificati. **La Figura 10.11a** fornisce una visione globale del ciclo di vita di un oggetto Libro: un libro viene ordinato, rimane in vita per un po' e alla fine viene eliminato o archiviato. Nella **Figura 10.11b** , lo stato vivo è ampliato per mostrare la sua struttura più fine. In questo esempio, lo stato viene raffinato in sottostati disgiunti *che* si escludono a vicenda: un libro è disponibile *o preso in* prestito. [1] Il passaggio dallo stato ordinato allo stato vivo è trascinato al confine dello stato vivo. Ciò equivale a una transizione allo stato iniziale all'interno dell'area grafica di vivo. Il passaggio dallo stato nidificato disponibile agli stati eliminati e archiviati viene effettuato direttamente. Per indicare questa transizione da uno stato interno soppresso di vivo a dismesso e archiviato nella **Figura 10.11a** , le transizioni non sono tracciate dal confine di vivo, ma da un cosiddetto **stub,** mostrato come una piccola linea verticale tracciata all'interno del suo confine.

10.3.3 Il diagramma di sequenza

Gli oggetti comunicano inviando messaggi. Per eseguire un determinato compito, potrebbe essere necessario scambiare una particolare sequenza di messaggi tra due o più oggetti. L'ordine temporale in cui deve verificarsi questa sequenza di messaggi può essere rappresentato in un **diagramma di sequenza.** Un diagramma di sequenza è un tipo di **diagramma di interazione.** Un secondo tipo di diagramma di interazione è il **diagramma di comunicazione,** discusso nella **Sezione 10.3.4** . Nel dominio delle telecomunicazioni, i diagrammi di sequenza sono noti come diagrammi di sequenza dei messaggi; forniscono una notazione standard per la progettazione e la specifica dei protocolli. Il diagramma di sequenza viene utilizzato anche nella comunità del design pattern, per rappresentare graficamente l'interazione tra due o più oggetti che partecipano a un design pattern.

In un diagramma di sequenza, la dimensione orizzontale mostra i vari oggetti che partecipano all'interazione. Un oggetto viene mostrato come una linea tratteggiata verticale, la sua "linea di vita". Il periodo in cui l'oggetto è attivo (all'interno della particolare sequenza di messaggi rappresentata) è rappresentato da un sottile rettangolo. Se la distinzione tra attivo e inattivo non è importante, l'intera linea di vita può essere mostrata come attivazione, come in **Figura 10.12** . L'ordine in cui vengono mostrati gli oggetti non ha significato.

La dimensione verticale indica la sequenza temporale dei messaggi. Di solito, solo L'*ordine* in cui vengono visualizzati i messaggi ha un significato. Per le applicazioni in tempo reale, l'asse del tempo può mostrare valori numerici effettivi.

I messaggi vengono visualizzati come archi etichettati da un oggetto all'altro. La disposizione verticale dei messaggi ne indica l'ordine. Le etichette possono anche contenere numeri di

sequenza, particolarmente utili per indicare la concorrenza. Un messaggio può anche essere etichettato con una guardia, un'espressione booleana che indica la condizione che deve valere per l'invio del messaggio.

La Figura 10.12 mostra una possibile sequenza di interazioni tra un utente, un catalogo di libri disponibili e un oggetto che gestisce le prenotazioni. Il primo messaggio arriva da una fonte esterna sconosciuta. Questo messaggio è chiamato **messaggio trovato.** L'utente invia quindi una richiesta al catalogo per cercare un determinato titolo. Il catalogo reagisce inviando i dati su quel titolo all'utente. Se il titolo non è disponibile (questo è indicato da un'espressione booleana, la guardia, tra parentesi quadre), una richiesta di riservare quel titolo viene inviata all'oggetto che gestisce le prenotazioni. Qualche tempo dopo, quel titolo tornerà disponibile e le prenotazioni verranno notificate. Le prenotazioni degli oggetti invieranno quindi un messaggio al catalogo per conservare quel libro e avviseranno l'utente che il titolo è ora disponibile. L'ordine di questi due messaggi è irrilevante, quindi portano lo stesso numero di sequenza. L'utente può ora prendere in prestito il titolo e la prenotazione corrispondente verrà rimossa.

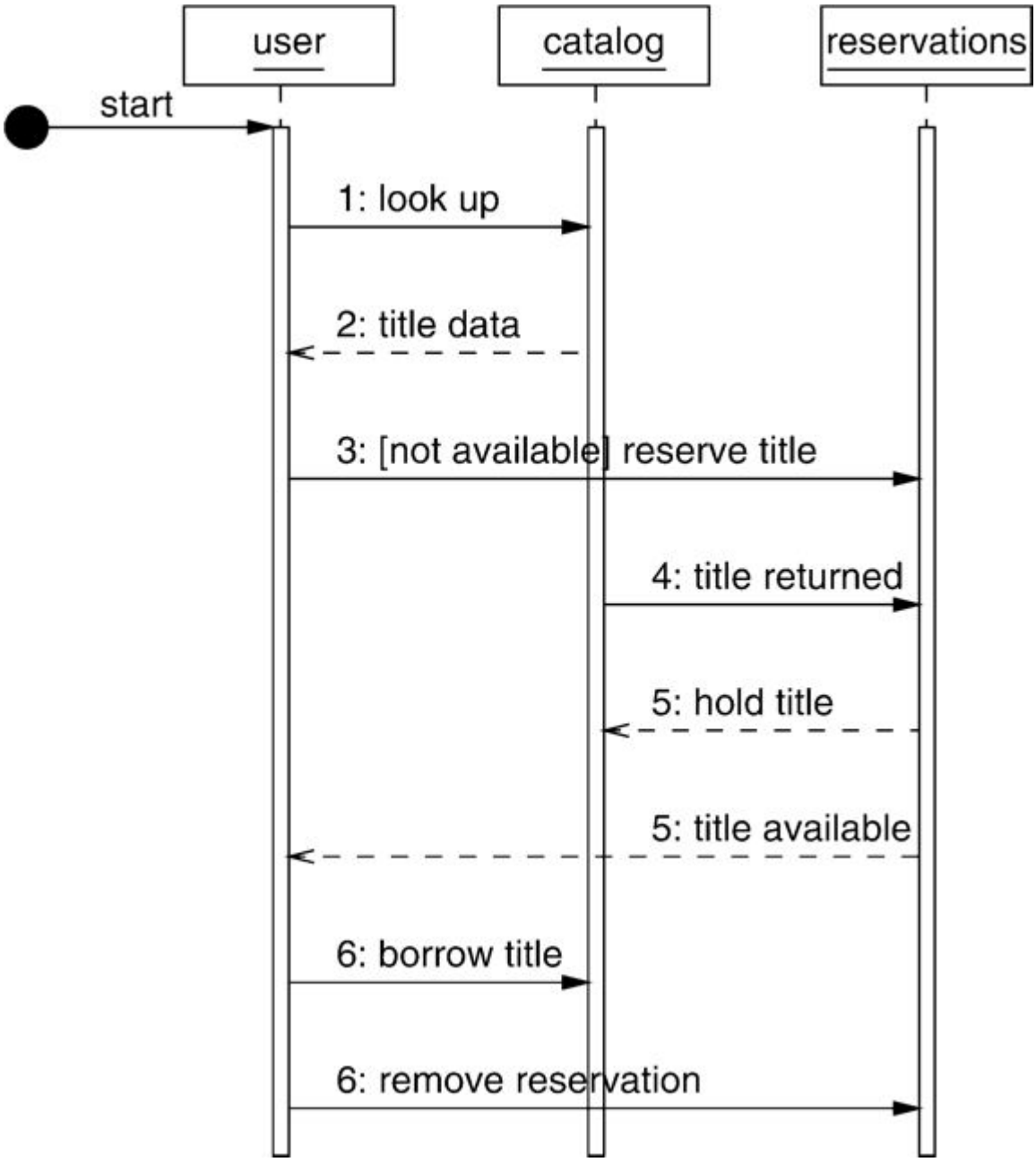


Figura 10.12 Diagramma di sequenza UML: prenotazione di un titolo

Anche in questo caso, UML ha un ricco vocabolario di notazioni per i diagrammi di sequenza. È possibile distinguere il passaggio di messaggi asincrono dal passaggio di messaggi sincrono, per indicare l'iterazione, per mostrare la creazione e la distruzione di oggetti e così via. Lo scopo principale del diagramma di sequenza rimane comunque lo stesso: una panoramica di facile lettura del passaggio di messaggi in una particolare sequenza di interazione.

10.3.4 Il diagramma di comunicazione

Il diagramma di comunicazione è un altro modo per mostrare uno scenario possibile per l'interazione tra un numero di oggetti correlati. Un diagramma di comunicazione è un grafo diretto in cui i nodi denotano entità e gli spigoli denotano la comunicazione tra tali entità.

Figura 10.13 mostra la stessa sequenza di interazioni dello scenario rappresentato nel diagramma di sequenza della **Figura 10.12** . I diagrammi di comunicazione enfatizzano gli oggetti e le loro relazioni rilevanti per una particolare interazione. Per fornire maggiori dettagli sull'interazione, gli attributi rilevanti possono essere mostrati all'interno dei nodi (aggiungendo un altro compartimento, come in un diagramma di classe) e questi attributi possono anche essere incorporati nelle etichette dei bordi.

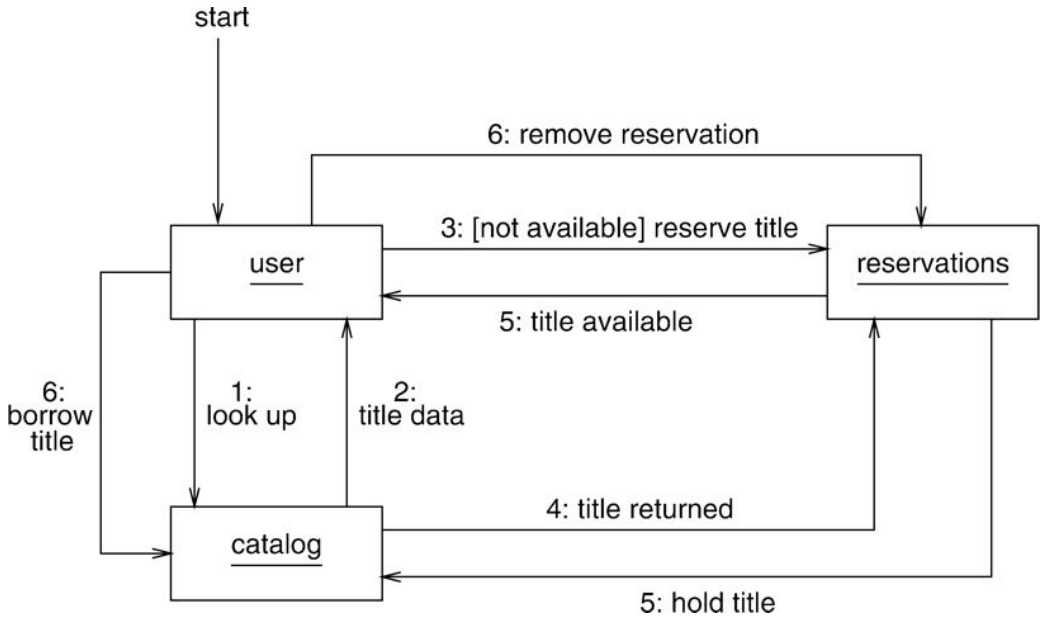


Figura 10.13 Diagramma di comunicazione UML: prenotazione di un titolo

I diagrammi di sequenza enfatizzano l'ordine dei messaggi e i numeri di sequenza sono opzionali; in un diagramma di comunicazione, i numeri di sequenza sono obbligatori poiché l'ordine non viene visualizzato graficamente.

10.3.5 Il diagramma dei componenti

Quando si progettano sistemi più grandi, può essere utile essere in grado di identificare entità più grandi di una singola classe. Questo può essere fatto in un **diagramma dei componenti.** Nelle descrizioni dell'architettura software, ad esempio, il diagramma dei componenti è un buon modo per rappresentare una vista del modulo di un sistema (vedere la **Sezione 11.3**).

In sostanza, un diagramma componente è un diagramma di classe con lo stereotipo «componente». In UML 1, il diagramma dei componenti aveva una forma speciale. In UML 2,

questo modulo è spesso rappresentato come un'icona di un piccolo componente all'interno del componente, come nella **Figura 10.14** . Oltre a questa icona, il diagramma dei componenti non introduce alcuna nuova notazione.

I componenti contengono classi o altri componenti. Nella **Figura 10.14** abbiamo modellato la pubblicazione come un componente contenente due classi, chiamate Searching e Storage. I componenti sono collegati da interfacce. La **Figura 10.14** usa la notazione 'balland-socket' per rappresentare le interfacce. Sia questa notazione che quella usata nella **Figura 10.9** sono consentite sia nei diagrammi delle classi che nei diagrammi dei componenti.

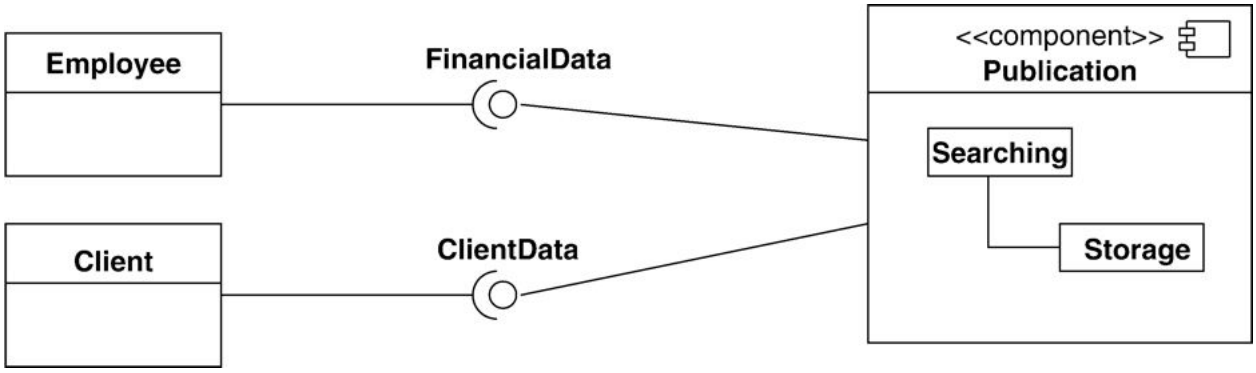


Figura 10.14 Diagramma dei componenti UML

10.3.6 Il caso d'uso

Una possibile tecnica di elicitazione dei requisiti è l'analisi basata su scenari; vedere anche **il capitolo 9** . Uno scenario è una storia che racconta come viene eseguita un'istanza di attività specifica. Spesso, scenari diversi sono variazioni sullo stesso tema. Ad esempio, uno scenario può descrivere il normale prestito di un libro, un altro può descrivere il prestito di un libro quando ci sono ancora multe in sospeso e così via. Un insieme di scenari con lo stesso obiettivo dell'utente, in questo caso il prestito, è chiamato **caso d'uso.**

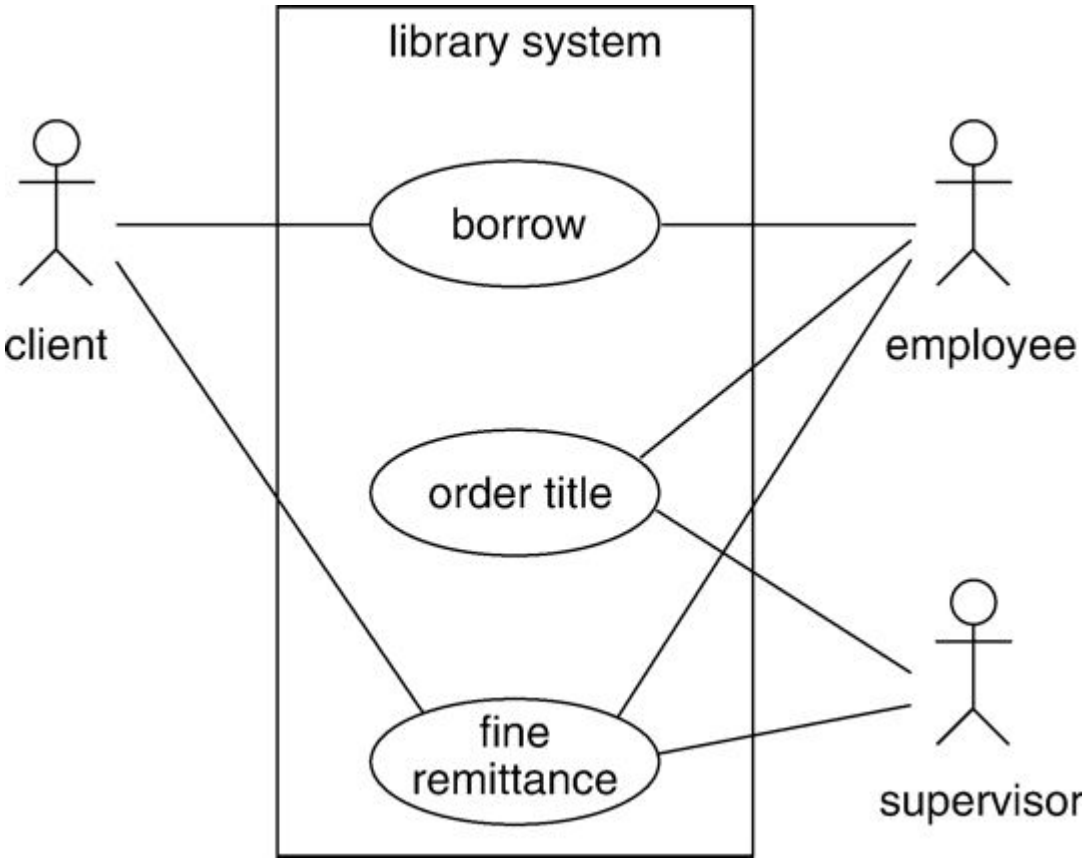
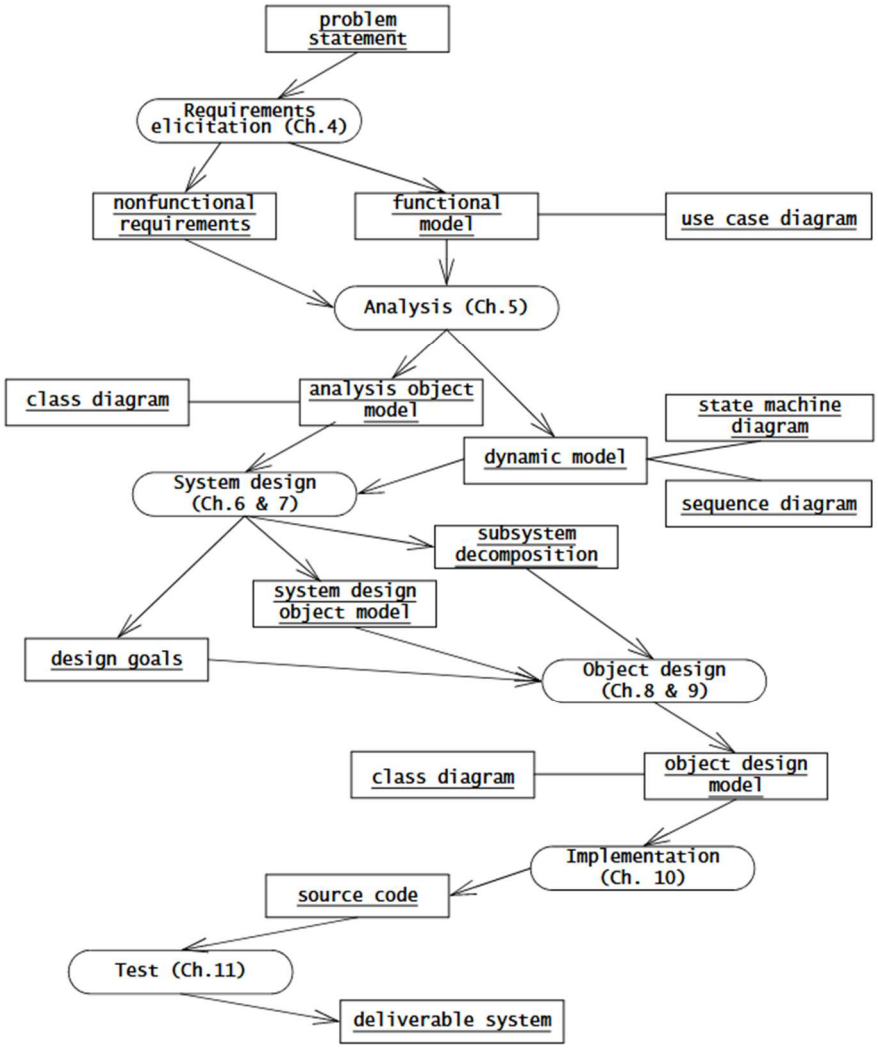


Figura 10.15 Diagramma del caso d'uso UML

Un caso d'uso può essere documentato in vari modi: come testo narrativo, utilizzando formalmente pre e postcondizioni, ad esempio, o graficamente come in un diagramma di transizione di stato. Il **diagramma** dei casi d'uso fornisce una panoramica di una serie di casi d'uso. Ciascun caso d'uso viene visualizzato come un'ellisse con il nome del caso d'uso. I casi d'uso sono racchiusi da un rettangolo che denota il confine del sistema. Un attore che inizia o partecipa a uno scenario viene mostrato come una figura stilizzata con il nome dell'attore di seguito. La **Figura 10.15** mostra parte del diagramma dei casi d'uso per il nostro sistema di librerie. Prendere in prestito un libro coinvolge due attori: un cliente e un impiegato della biblioteca. Molti altri casi d'uso coinvolgeranno anche questi due attori. L'ordinazione di un nuovo libro necessita dell'approvazione di un supervisore, così come la rimessa di una multa.

10.4 USO DI UML NEL CICLO DI VITA DEL SOFTWARE



E' possibile mappare l'uso dei diagrammi UML all'interno del processo di sviluppo del software. Essi si usano in varie **fasi** del ciclo di vita del software per rappresentare visivamente la struttura, il comportamento e le interazioni del sistema. Non sono legati a un singolo metodo di sviluppo (possono essere usati sia in approcci tradizionali come il *waterfall*, sia in quelli agili), ma servono come **strumento di comunicazione e analisi** tra analisti, sviluppatori, clienti e tester.

Ecco come vengono tipicamente impiegati nelle diverse fasi:

1. Analisi dei requisiti

In questa fase i diagrammi UML aiutano a **comprendere e documentare ciò che il sistema deve fare**.

- **Diagrammi dei casi d'uso (Use Case Diagram)**: mostrano le funzionalità del sistema dal punto di vista dell'utente (gli attori e le loro interazioni con il sistema).
- **Diagrammi di attività (Activity Diagram)**: rappresentano i flussi di lavoro o processi aziendali.
- **Diagrammi di stato (State Machine Diagram)**: descrivono il comportamento dinamico di oggetti che cambiano stato.

Servono per chiarire i requisiti funzionali e comunicare in modo chiaro con gli stakeholder.

2. Progettazione (Design)

Durante la progettazione, UML viene usato per **definire l'architettura e la struttura interna del sistema**.

- **Diagrammi delle classi (Class Diagram)**: descrivono le classi, i loro attributi, metodi e relazioni (ereditarietà, associazioni, ecc.).
- **Diagrammi dei pacchetti (Package Diagram)**: mostrano come è organizzato il codice in moduli o componenti.
- **Diagrammi di sequenza (Sequence Diagram)**: illustrano le interazioni tra oggetti nel tempo per realizzare una specifica funzione.
- **Diagrammi di comunicazione (Communication Diagram)**: alternative più compatte ai diagrammi di sequenza, mostrano i messaggi tra oggetti.
- **Diagrammi di componenti (Component Diagram)**: rappresentano i moduli software e le loro dipendenze.
- **Diagrammi di distribuzione (Deployment Diagram)**: mostrano dove i componenti software saranno installati fisicamente (server, dispositivi, ecc.).

Servono per passare da una descrizione concettuale a una progettazione tecnica chiara e coerente.

3. Implementazione

Durante la codifica, i diagrammi UML fungono da **guida per gli sviluppatori**:

- I **diagrammi delle classi** possono essere tradotti direttamente in codice (molti IDE supportano la generazione automatica).
- I **diagrammi di sequenza** aiutano a capire la logica di esecuzione e la collaborazione tra oggetti.

In questa fase, UML mantiene la coerenza tra modello e codice, specialmente nei progetti orientati agli oggetti.

4. Verifica e validazione (Testing)

UML aiuta a **progettare i test e verificare la correttezza del sistema**.

- I **casi d'uso** forniscono la base per definire i test funzionali.

- I **diagrammi di attività** possono essere usati per generare scenari di test automatici.
- I **diagrammi di stato** aiutano a coprire tutti i possibili comportamenti di un oggetto.

In questa fase UML contribuisce a una copertura più sistematica dei test.

5. Manutenzione ed evoluzione

Durante la manutenzione, i diagrammi UML servono a **documentare il sistema** e a **facilitare modifiche e aggiornamenti**.

- Consentono di capire rapidamente l'architettura esistente e l'impatto delle modifiche.
- I modelli UML aggiornati diventano parte della documentazione viva del progetto.

Fase	Scopo dell'UML	Tipi di diagrammi principali
Analisi	Capire e specificare i requisiti	Casi d'uso, attività, stato
Progettazione	Definire architettura e interazioni	Classi, sequenza, componenti, distribuzione
Implementazione	Guidare la codifica	Classi, sequenza
Test	Derivare e documentare casi di test	Casi d'uso, attività, stato
Manutenzione	Documentare e aggiornare il sistema	Tutti i precedenti

10.5 RIASSUNTO

Durante l'ingegneria dei requisiti e la progettazione, vengono applicate una varietà di notazioni di modellazione. La maggior parte di questi utilizza una sorta di diagramma box-and-line. Le notazioni di modellazione tradizionali di oggi derivano dall'Unified Modeling Language (UML). Molti diagrammi UML sono basati o derivati da tipi di diagramma precedenti. In questo capitolo, abbiamo discusso una selezione di notazioni di modellazione classiche e dei principali tipi di diagramma UML.

Le classiche notazioni di modellazione discusse sono:

- modellazione entità-relazione (ERM), utilizzata per modellare la struttura dei dati,
- macchine a stati finiti (FSM), utilizzate per modellare stati e transizioni di stato di un sistema,
- diagrammi di flusso di dati (DFD), utilizzati per modellare la scomposizione funzionale rispetto ai dati,
- Carte CRC, una semplice notazione per documentare le decisioni di progettazione collaborativa.

UML si è evoluto da precedenti metodi di analisi e progettazione orientati agli oggetti. I concetti utilizzati in UML, come oggetto, attributo, classe e relazione, hanno origine nel campo dell'orientamento agli oggetti. UML 2 offre 13 tipi di diagramma che rientrano in due classi. Alcuni diagrammi forniscono una vista statica del sistema. Ad esempio, un diagramma di classe mostra come un sistema è organizzato staticamente in classi. Altri diagrammi danno una vista dinamica. Ad esempio, un diagramma di sequenza mostra l'ordine temporale degli scambi di messaggi tra istanze di classi.

10.6 ULTERIORI LETTURE

La modellazione entità-relazione è stata sperimentata da Chen (**1976**). Molti testi sulla modellazione di database includono un'elaborata discussione sull'ERM; si veda, ad esempio, (Batini *et al.*, **1992**). I diagrammi di stato sono descritti in (Harel, **1988**). Le carte CRC sono descritte in (Beck e Cunningham, **1989**).

Le diverse visioni della nozione di oggetto sono discusse in (Taivalsaari, **1993**). I vari significati di attributo e le relative nozioni come aggregato, parte e membro sono discussi in (Motschnig-Pitrik, **1996**). Wegner (**1992**) è un classico articolo sulle varie dimensioni della modellazione orientata agli oggetti. Fowler (**2004**) fornisce una buona introduzione a UML. UML è ampiamente discusso in due libri dai suoi creatori, (Booch *et al.*, **1999**) e (Rumbaugh *et al.*, **1999**). L'uso effettivo di UML è riportato in (Dobing and Parsons, **2006**), (Lange *et al.*, **2006**) e (Grossman *et al.*, **2005**).

Esercizi

1. Spiegare i seguenti concetti dalla modellazione entità-relazione: entità, tipo di entità, valore dell'attributo, attributo, relazione.

2. Definire i seguenti termini: oggetto, stato, attributo, messaggio ed ereditarietà.

3. Spiegare la differenza tra la relazione specializzazione-generalizzazione e la relazione parte intera.

4. Spiegare la differenza tra un diagramma di classe e un diagramma di macchina a stati.

5. Spiegare la differenza tra un diagramma di sequenza e un diagramma di comunicazione.

6. Spiegare la differenza tra un diagramma delle classi e un diagramma dei componenti.

7. A cosa servono le schede CRC e gli scenari di casi d'uso nell'analisi e nella progettazione orientata agli oggetti?

8. In quali aspetti un diagramma di stato UML differisce da un diagramma di transizione di stato?

Capitolo **Errore. Per applicare Heading 1 al testo da visualizzare in questo punto, utilizzare la scheda Home. Errore. Per applicare Heading 1 al testo da visualizzare in questo punto, utilizzare la scheda Home.**

9. ♡In che senso l'interfaccia verso una classe può essere considerata un contratto? Quali sono le ripercussioni di ciò per le relazioni di sottotipizzazione? (Vedi (Meyer, **1992**).)

[1] UML consente inoltre di perfezionare uno stato in sottostati simultanei. Ad esempio, quando un libro viene restituito, è necessario fare diverse cose. È necessario verificare se il libro viene restituito entro il tempo stabilito. In caso contrario, potrebbe essere dovuta una multa. Anche eventuali prenotazioni in sospeso devono essere controllate e, in tal caso, è necessario gestire una di queste prenotazioni. Questi processi secondari possono essere gestiti contemporaneamente. Può esserci un libro di ritorno dello stato che, una volta perfezionato, si traduce in due o più sottostati simultanei e correlati. Ciò viene mostrato affiancando la regione grafica dello stato utilizzando linee tratteggiate per separare le sottoregioni.